利用源文本与字段检查修复文档知识图谱

Hua Peng[1†], Zifu Tao[2†], Yu Guo[1], Qiang Zhou[2],
Tian Zhou[3*], Bo Han[3]

[1]中移系统集成有限公司, 中国.
[2]同济大学, 中国.
[3]西安交通大学, 中国.

*通讯作者及邮箱： tianzhou@xjtu.edu.cn;
其他作者邮箱： penghua@cmict.chinamobile.com;
2253323@tongji.edu.cn; guoyu1@cmict.chinamobile.com;
zhouqiang@cmict.chinamobile.com; bohan@xjtu.edu.cn;
†以上作者对本文贡献相同。

摘要

从文本构建的知识图谱常有事实缺失、取值错误和三元组重复。修复需要找出这些错误，用约束引导修改，并选择有用的编辑。我们提出一个框架，将多尺度质量评估、约束驱动修复和尺度感知路由结合起来。局部、图和来源检查形成共享的评估概况。来源与字段约束引导图生成，并过滤候选三元组。神经路由器根据评估概况为候选编辑赋予权重。选择器在图的副本上检查每项编辑，在图约束下比较质量收益和成本，并在接受修改后更新评估概况。实验覆盖中文政务、金融和环境记录、收据及合同，并包含组件测试和人工评价。主流程修复了 98% 的注入缺陷。在收据任务中，来源行提示使 F1 比 简单流程 高 4.76 个百分点。评价同时记录恢复的事实和保留的原正确事实，展示各修复阶段如何影响输出图。

关键词: 知识图谱修复，源文本，字段检查，文档抽取

中文完整译本，对应英文合并版。参考文献保留原始出版信息；模型标识与程序名称保留原文。图内英文标注的中文对照见译文附页。

1 引言

知识图谱（KG）连接不同来源的事实，支持问答、推荐和数据分析(Ji et al. 2022; Pan et al. 2023)。这些应用依赖所存事实的质量。关系缺失会使有用的信息无法被发现，错误取值则可能导致错误答案或决策。随着更多图谱从文本构建，检查和提高

图谱质量已成为知识图谱构建的重要部分(Zaveri et al. 2016; Paulheim 2017)。其目标是恢复有用事实，并保留图中已有的正确信息。

从文本抽取事实后，同一张图中会留下多类错误。来源噪声和含义不清的表述可能造成字段缺失、事实重复、关系错误，以及取值被放入错误字段(Ji et al. 2022; Paulheim 2017)。这些错误将图结构与源文本的含义联系起来。例如，收据上可能同时出现小计和最终金额，但两者的作用不同。判断某个金额是否属于总额字段，需要检查它是否出现在原文中，也需要判断它的含义。因此，图修复需要同时检查结构和字段取值的来源证据。

现有方法从不同方面提高图谱质量。基于规则的检查发现违反图规则和字段要求的情况(Kontokostas et al. 2014; Knublauch and Kontokostas 2017)。嵌入模型根据图中的模式，为事实成立的可能性打分(Bordes et al. 2013; Nickel et al. 2016)。大语言模型（LLM）利用文本和图上下文提出修复建议(Pan et al. 2023; Lin et al. 2025)。每类方法都为修复决策提供有用信息。规则违规指出了未满足的要求，模型建议则给出可能的替换内容。将这些信号连接起来，需要统一描述错误、明确修改边界，并在修复动作之间作出选择。

质量错误出现在不同尺度。局部检查可以发现重复事实，图检查则检验关系和字段间的冲突。来源检查将保存的取值与文档证据联系起来(Zaveri et al. 2016; Wang et al. 2021)。一个值可能满足字段规则，却指向文档中的错误位置。修复需要一种视图，同时说明错误发生在哪里，以及需要什么证据来修正它。

修复还需要在恢复缺失事实与保留正确事实之间作出权衡。添加事实可能引入字段冲突，删除事实则可能损失有用信息。一次修改也可能影响图的其他部分。例如，删除一条边可能使一个正确节点失去连接。因此，修复需要为修改设定清楚的边界，并为每项编辑计入成本。

修复动作的选择需要反映错误类型。简单规则可以删除重复事实，并修正明确的结构错误。缺失值或错误值则需要阅读源文本。多个编辑建议可能针对同一字段，也可能改变另一项编辑的效果。执行顺序很重要，因为一次被接受的修改可能改变下一步需要修复的内容。选择过程应使用当前错误、预期收益和编辑成本。

我们的思路是通过共享的评估概况，将这些决策联系起来。评估概况记录错误和质量分数，约束设定修改边界，编辑选择则利用两者引导修复。三者共同将错误类型与修复范围及成本联系起来。围绕这一联系，我们构建一个结合多尺度评估、约束驱动修复和尺度感知路由的框架。

我们用覆盖实体、图和上下文尺度的*多尺度质量评估*支撑这一思路。实体检查发现局部错误，图检查检验字段和关系规则，上下文检查使用原始文档。这些检查形成共享的评估概况，记录不同范围内的错误和质量分数。在主路径中，错误报告引导模型提出修改建议。编辑选择器利用质量分数比较试改。这将发现的问题与修复过程联系起来。

*约束驱动优化*为修改设定清楚的边界。主修复路径结合规则清理、大语言模型生成和三元组过滤。来源与字段要求决定哪些生成的三元组可以进入输出。字段含义和来源行提示引导模型寻找有用证据。在编辑选择中，质量下界和密度上限限定允许的试改图，编辑成本则鼓励小幅修改。这些要求将修复目标与接受修改建议时使用的检查联系起来。

*尺度感知路由*支持一条逐次选择编辑的路径。小型神经模型利用评估概况，为实体、图和上下文尺度的编辑赋予权重。选择器在图的副本上测试建议编辑，在图约束下比较质量收益与成本。接受一项编辑后，它再次检查图并更新评估概况。这样，下一次决策就能使用修改后的图和剩余错误。

本文有三项贡献：

- **多尺度质量评估。** 我们将局部结构、图规则和来源检查汇入共享概况，将发现的缺陷与修复决策联系起来。
- **约束驱动修复与尺度感知路由。** 我们为生成的图定义来源和字段约束。神经路由路径为候选编辑赋予权重，并在图边界内比较其质量收益和成本。
- **文档图谱评价。** 我们在注入缺陷、模型输出、收据和合同上比较修复方法及其组件，并开展人工评价。保存的输出和决策日志支持检查各个修复阶段。

实验表明，主流程修复了中文记录中 98% 的注入缺陷，来源行提示使收据任务的 F1 比 简单流程 高 4.76 个百分点。

2 相关工作

2.1 图构建与抽取事实检查

图构建将来源数据转为相互连接的事实(Ji et al. 2022)。DBpedia、YAGO 和 Freebase 从结构化及部分结构化来源中建立知识库(Auer et al. 2007; Suchanek et al. 2007; Bollacker et al. 2008)。质量检查帮助这一过程选择事实。知识融合汇总多个来源的证据，为候选事实打分(Dong et al. 2014)。这将抽取过程与保留哪些事实的问题联系起来。

学习方法利用图中的模式预测链接(Bordes et al. 2013; Wang et al. 2017; Schlichtkrull et al. 2018)。相关综述介绍了大语言模型如何利用文本构建、补全和整合知识图谱(Pan et al. 2023, 2024; Bian 2025)。推理提示和提示模式引导模型回答(Wei et al. 2022; White et al. 2023)。检索增强生成将来源文本加入模型输入(Lewis et al. 2020)。Neo4j Labs 的 LLM Knowledge Graph Builder 将此类模型用于文档，并支持用户设置模式(Neo4j Labs 2024)。这些研究提供了生成候选事实的方法。本文从已抽取的图出发，利用源文档检查建议的字段值。

2.2 图质量与检查范围

质量评估关注所存事实是否正确、完整、一致且及时(Zaveri et al. 2016; Paulheim 2017; Wang et al. 2021)。一类方法检查数据与已有模式的符合程度。统计方法根据取值分布查找异常值(Wienand and Paulheim 2014; Paulheim and Bizer 2014)。嵌入分数衡量三元组与学到的图模式之间的符合程度(Nickel et al. 2016)。另一类方法通过测试和图约束，明确有效记录应满足的要求(Kontokostas et al. 2014; Knublauch and Kontokostas 2017)。人工评价有助于判断答案不明确的情况(Acosta et al. 2013)。

这些方法将质量目标转为具体检查，用于定位不同错误。对于文档图谱，局部结构、字段规则和来源证据从相互关联的角度检查同一条记录。本文的评估概况将事实重复、字段违规和来源不符汇入一份报告，将发现的问题与修复选择联系起来。

2.3 图修复与动作选择

修复需要恢复缺失事实，并保留正确信息。补全方法利用路径和嵌入预测缺失链接(Lao et al. 2011; Bordes et al. 2013; Yang et al. 2015; Shi and Weninger 2017)。神经模型也学习为可能的链接打分(Dettmers et al. 2018; Zhu et al. 2021)。基于路

径的强化学习支持查询回答(Das et al. 2018)，少样本学习则根据少量样本预测链接(Chen et al. 2019)。这些方法为图谱补全提供候选事实。

规则为建议的事实提供可检查的结构。规则挖掘从知识库中发现正向和负向模式(Galárraga et al. 2013; Ortona et al. 2018)。嵌入训练可以加入逻辑规则和简单约束(Guo et al. 2016; Ding et al. 2018)。RNNLogic 和 pLogicNet 将逻辑规则与学习得到的预测结合(Qu et al. 2021; Qu and Tang 2019)。这些研究将事实预测与引导学习和检查的规则联系起来。

语言模型通过提示和上下文提供另一种修复方式。Lin 等人将被违反的 SHACL 约束和图上下文加入修复提示(Lin et al. 2025)。ReAct 将模型推理与信息收集动作结合(Yao et al. 2023)。本文据此设计 SHACL 上下文和 ReAct-style 基线。在这些研究的基础上，我们将来源证据与字段约束、编辑选择联系起来。主路径过滤候选三元组，尺度感知选择器则在图约束下，根据质量收益与成本比较试改。

3 任务与方法

修复方法需要将发现的错误与执行的修改联系起来。重复事实需要局部修改，错误字段值需要查阅来源，每项修改还必须满足图规则。我们用评估概况连接这些决策，记录实体、图和上下文三个尺度的问题，再用约束规定允许的输出，并根据错误报告和源文本作出修复决策。

3.1 任务与整体设计

本文研究关系表示字段的文档图。输入包括三元组列表 G、源文本 X、固定文档编号 h_0 和允许关系 $\mathcal{R}_X$。列表可以包含同一三元组的多个副本。目标图中，每种关系最多对应一个非空值，字段为可选项。文档记录和收据符合这一形式。修复根据这些输入生成图 $\widehat{G}$，评分器将其与参考图 G^* 比较。

框架包含两条分别评价的修复路径。主路径用错误报告引导模型生成完整图，再用过滤器依据来源和字段规则检查生成的三元组。编辑选择路径用质量分数逐项比较修改，路由器为编辑范围分配权重，试改检查衡量收益和成本。两条路径都将评估连接到修复：报告引导取值生成，分数引导编辑选择。

3.2 多尺度质量评估

检测不同错误需要不同信息。重复三元组可以在图内找到，复制的值则需要与文档核对。因此，我们按范围组织检查。实体检查覆盖局部连接和重复事实；图检查覆盖文档编号、字段名和关系规则；上下文检查将取值与源文本比较。这里，上下文指来源文档。这些检查共同构成评估概况，指出问题的类型和范围。图 1 展示这些检查组。

3.2.1 字段检查与错误报告

清理首先用明确的规则修正错误。预处理器 P 删除重复三元组。若一条边的对象为 h_0，而主体是其他节点，则将该边反向。随后删除头节点或关系不合法的三元组。这里，头节点就是三元组的主体。清理后的图为 $G_s = P(G; h_0, \mathcal{R}_X)$。

字段检查分别识别结构错误和来源错误。对三元组 $t = (h, r, o)$，检查为

$$H(t) = \mathbf{1}[h = h_0], \quad R(t) = \mathbf{1}[r \in \mathcal{R}_X], \quad S(t; X) = \mathbf{1}[o \neq \emptyset \ \wedge \ o \text{ 出现在 } X]. \tag{1}$$

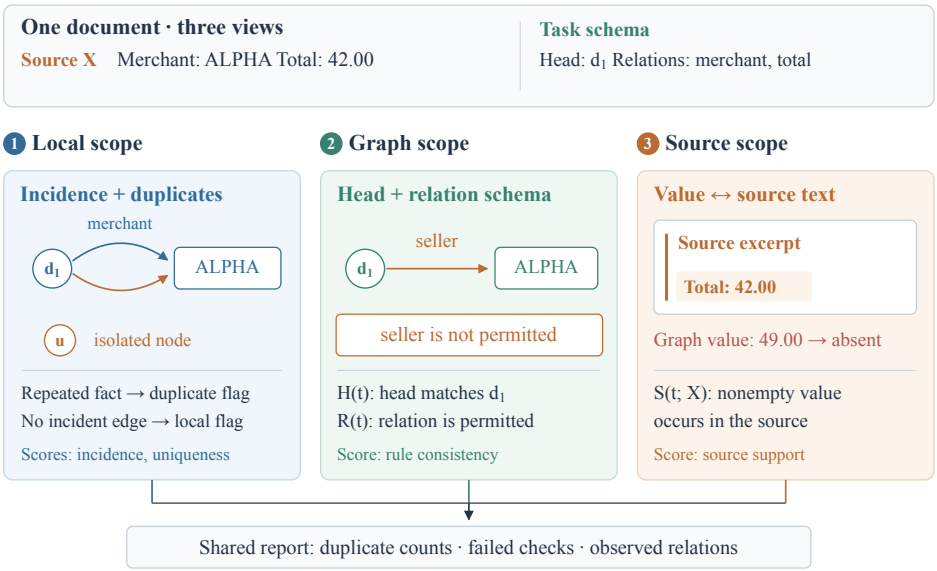


图 1 图检查的三个范围。局部检查查找重复事实和无连接节点，图检查测试文档编号和字段名，来源检查在文本中查找值。两条修复路径都用这些检查构建评估概况。

H 检查文档编号，R 检查关系，S 检查来源。一个值可以通过前两项检查，却未通过第三项。分别检查可以清楚说明每条三元组被拒绝的原因。

主路径将这些检查汇入错误报告：

$$D(G_s, X) = \big(n_{\text{dup}}, I_H, I_R, I_S, \mathcal{R}(G_s)\big). \tag{2}$$

报告保存重复数、未通过检查的位置和已出现的关系。例如，$I_S = \{i : S(t_i; X) = 0\}$ 列出未通过来源检查的位置。报告基于清理后的输入构建。失败位置指出需要复核的取值，已出现的关系表明图中已有的字段。模型同时接收报告、图和源文本，使生成步骤获得明确的问题说明。

3.2.2 用于试改图的质量分数

选择编辑还需要比较修改前后的图。为此，我们使用四项分数：节点连接、三元组唯一性、规则一致和来源支持。

$$\begin{aligned} \mathbf{s} &= (Q_{\text{inc}}, Q_{\text{uniq}}, Q_{\text{rule}}, Q_{\text{src}}), \\ Q_i &= 100(1 - r_i), \quad Q = \tfrac{1}{4}\sum_i Q_i. \end{aligned} \tag{3}$$

每个 r_i 是对应检查的错误率。对于非空图，各错误率为

$$r_{\text{inc}} = \frac{I}{n}, \quad r_{\text{uniq}} = \frac{m - c}{m}, \quad r_{\text{rule}} = \frac{B}{m}, \quad r_{\text{src}} = \frac{V}{m}. \tag{4}$$

I 为无连接节点数，c 为三元组相似组数。B 为被图规则标记的不同三元组数。V 为去掉空白后仍未在文本中出现的值的数量。这里，n、m 分别为节点数和三元组数。四项分数使用同一尺度，其均值给出试改质量。选择器也保留各项分数，从而检查编辑是否以降低一项分数为代价提高另一项。

缺失字段没有可供检查的三元组，因此需要额外检查。评估概况还报告源文本中存在而图中缺少的字段。这些报告进入违规计数和候选列表。字段报告和质量分数共同说明需要修复的位置，以及试改对图的影响。

3.3 约束驱动的图修复

生成的图需要选择字段值，并满足任务规则。我们将这两项工作分别交给生成和过滤步骤。模型利用来源和错误报告提出取值，过滤器再对候选应用固定的来源与字段检查。这样，描述输入错误与决定保留哪些输出三元组使用同一组要求。

3.3.1 根据源文本生成图

规则清理处理明确的结构错误。恢复缺失值或替换错误值需要阅读来源，语言模型执行这一步并返回有序三元组列表：

$$C = L_\theta(X, h_0, \mathcal{R}_X, G_s, D(G_s, X)). \tag{5}$$

提示要求生成完整图，复制来源值，保留正确字段，并补充或修正缺失、错误的值。返回完整图使模型可以一起考虑各字段。输出顺序也为过滤时在竞争取值之间作出选择提供固定依据。

3.3.2 字段含义与来源行

源文本可能包含多个符合相同字段格式的值。收据可能同时列出品牌和公司，或小计和最终金额。我们为每个字段提供简短含义，让模型据此选择符合字段作用的值。所有收据修复方法都获得这些含义和带行号的原文。

来源行索引提示各字段可能对应的证据位置。固定模式查找公司词、地址词、邮政编码、日期和总额标签。每条匹配行都是锚点，索引包含该行、前一行及后两行。当标签和值位于不同行时，邻近行可提供上下文。简单流程 获得完整原文和字段含义；证据索引 还获得行提示，同时保留完整原文。

3.3.3 检查生成的三元组

过滤器将任务规则转成输出要求。三元组必须使用给定文档编号和允许关系，其值必须出现在来源中，每个字段最多保留一个值。令 $\mathrm{uniq}(C)$ 按输出顺序去重。通过检查的三元组为

$$C_X = \{t \in \mathrm{uniq}(C) : H(t)R(t)S(t;X) = 1\}. \tag{6}$$

允许的输出集合为

$$\mathcal{F}(C_X) = \left\{ A \subseteq C_X : \sum_{t \in A} \mathbf{1}[r_t = r] \le 1,\ \forall r \in \mathcal{R}_X \right\}. \tag{7}$$

算法 1 利用源文本和字段检查进行修复

输入：　输入三元组 G、原文 X、文档编号 h_0、允许关系 $\mathcal{R}_X$
输出：　修复图 $\widehat{G}$ 和拒绝日志 J

$G_s \leftarrow P(G; h_0, \mathcal{R}_X)$
$D \leftarrow D(G_s, X)$
$C \leftarrow \mathrm{Parse}(L_\theta(X, h_0, \mathcal{R}_X, G_s, D))$
$\widehat{G} \leftarrow \emptyset$; $J \leftarrow \emptyset$; $B \leftarrow \emptyset$
对 按生成顺序遍历候选 $t = (h, r, o)$ 执行
    若 t 之前已在 C 中出现 则
        将 $(t, \text{重复})$ 加入 J
    否则 若 $H(t)R(t)S(t; X) = 1$ 且 $r \notin B$ 则
        $\widehat{G} \leftarrow \widehat{G} \cup \{t\}$; $B \leftarrow B \cup \{r\}$
    否则
        将 $(t, \text{未通过检查})$ 加入 J
    结束 若
结束 对
返回 $\widehat{G}, J$

式 6 规定证据和模式要求，式 7 规定字段数量上限。第一步删除未通过检查的三元组，第二步处理同一字段的多个竞争取值。

过滤器按顺序扫描候选，保留每个字段的第一个有效值。输出 $\widehat{G}$ 的大小为

$$|\widehat{G}| = \max_{A \in \mathcal{F}(C_X)} |A| = |\{r : \exists t \in C_X,\ r_t = r\}|. \tag{8}$$

每个字段构成一组，每组最多放一条三元组。从每个有有效值的组中保留一个候选，即可得到允许范围内最大的输出数量。同一字段内按输出顺序取舍。因此，生成步骤选择取值并给出顺序，过滤步骤执行来源匹配、模式规则和单值要求。

收据过滤器与解析器使用相同空白规则。$N(s)$ 将连续空格或换行变成一个空格，并去掉首尾空白。来源检查为

$$S_{\mathrm{ws}}(t; X) = \mathbf{1}[|N(o)| > 0 \ \wedge\ N(o) \text{ 出现在 } N(X)]. \tag{9}$$

这一处理保留词边界、大小写和标点。收据后续测试使用该检查，早期测试使用原来的精确文本检查。

算法 1 将这些步骤组合为 诊断＋门控。诊断 指错误报告，门控 指三元组过滤器。主路径对每篇文档调用模型一次。过滤器同时返回修复后的图和被拒绝三元组的日志，使每项删除都能追溯到具体检查。

3.4 尺度感知路由与编辑动作

一项编辑会改变接下来需要修复的内容。多个候选可能针对同一字段，一项修改也可能使另一项修改不再需要。因此，编辑选择路径在每次接受修改后重新检查图。尺度感知路由根据当前评估概况为局部、图和来源编辑赋权，试改检查再比较收益和成本，并选择动作。图 2 展示这条路径。

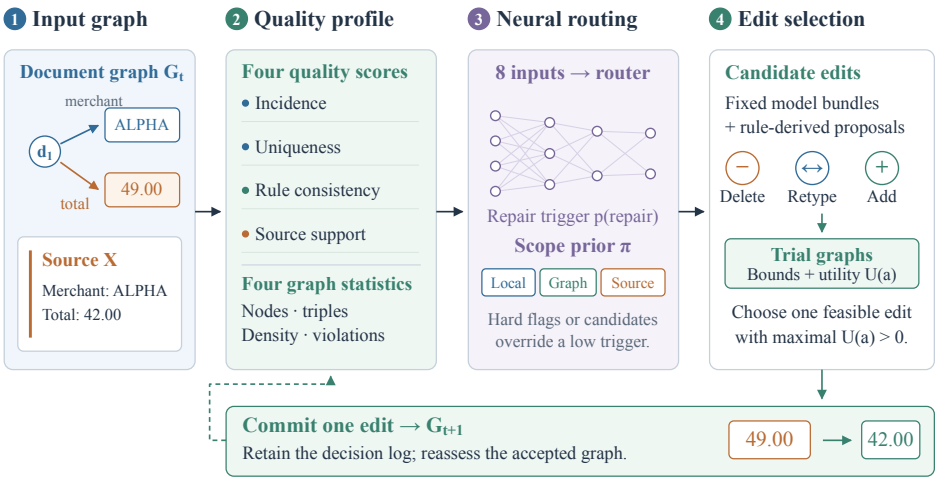


图 2 用于编辑选择的尺度感知路由。评估概况和图统计量输入路由器，得到修复分数和三个编辑范围的权重。选择器在图副本上尝试模型与规则编辑，执行最佳允许正分编辑，再次检查。

3.4.1 根据评估概况进行路由

路由器既需要错误分数，也需要待修复图的规模信息。它接收四项质量分数和四个图统计量：

$$\mathbf{g} = (n, m, \rho, n_{\text{viol}}), \qquad \rho = \frac{|E_{\text{pair}}|}{n(n-1)} \quad (n > 1). \tag{10}$$

这里，n、m 分别为节点数和三元组数。E_{pair} 包含不同的有向节点对，不含自连接。$n \leq 1$ 时密度为零。n_{viol} 是报告的违规数。这些统计量与 $\mathbf{s}$ 中的比例分数一起描述图规模和已报告的问题。字段报告 D 向主路径提供错误位置，选择器则使用这一数值概况为编辑范围赋权。

路由器将这八个输入映射为修复分数和范围权重：

$$(p_{\text{repair}}, \boldsymbol{\pi}) = f_\phi(\text{scale}([\mathbf{s}; \mathbf{g}])). \tag{11}$$

scale 使用训练集均值和标准差。$\boldsymbol{\pi}$ 有三个非负权重，合计为一，分别对应实体、图和上下文编辑。修复分数帮助决定何时开始试改检查，范围权重进入候选评分，将当前错误概况与下一项编辑的选择联系起来。

3.4.2 候选编辑与成本

规则和模型建议提供候选编辑。选择器可以删除三元组、改变关系或添加三元组。改变关系在代码中称为 retype。相关编辑可以组成一个包并一起试验，使选择器在多项操作影响同一字段时评估整体修改。

编辑成本使试改评分偏向较小的修改。删除已有三元组的成本高于改变关系或添加三元组。成本为

$$c(a) = \begin{cases} 0.30 & \text{删除,} \\ 0.16 & \text{改关系,} \\ 0.06 & \text{添加.} \end{cases} \tag{12}$$

编辑包的成本是其中各项编辑成本之和。候选必须在试改评分中获得足够收益来抵消成本，从而将修改数量和类型纳入修复决策。

3.5 试改检查与修复循环

编辑的作用取决于它修改的图。因此，我们在图副本上试做每个候选，并重新计算完整评估概况。令 G_a 为试改图，h 为硬规则错误数，$\sigma(a)$ 为编辑范围，$z(a)$ 为候选置信度。默认分数为

$$\begin{aligned} U(a) = {} & \frac{Q(G_a) - Q(G)}{100} + 0.20\frac{\max(0, h(G) - h(G_a))}{\max(1, h(G))} \\ & - 0.35c(a) + 0.05\log\bigl(\max(\pi_{\sigma(a)}, 0.05) + 10^{-6}\bigr) + 0.02z(a). \end{aligned} \tag{13}$$

前两项奖励质量提高和规则错误减少，其余项计入编辑成本、范围权重和候选置信度。这些项将候选修改与当前图比较，并用一个分数表达其收益和成本。

平均质量提高时，其中一项分数可能下降。因此，我们既逐项检查分数下界，也限制分数下降幅度。已经满足的下界必须继续满足，低于下界的分数必须保持或提高。我们还限制图密度，并要求非空图保持非空。这些要求确定允许的试改集合 $\mathcal{A}_t^{\mathrm{ok}}$。选择步骤为

$$a_t^* = \arg\max_{a \in \mathcal{A}_t^{\mathrm{ok}}} U(a), \qquad U(a_t^*) > 0. \tag{14}$$

式 14 在当前试改编辑中作出选择。边界决定哪些编辑允许执行，分数为允许编辑排序。没有允许编辑获得正分时，图保持原样。

算法 2 连接评估、路由和执行。接受编辑后，下一轮使用修改后的图和新的评估概况。即使修复分数较低，只要存在已提供的候选或硬规则错误，也会开始试改检查。当没有有用的允许编辑、不再需要试改，或达到步数上限时，循环结束。日志保存试改图、分数、决策和停止原因。

4 实现

以下设置说明评估、生成和编辑选择步骤如何运行，依次给出评分规则、路由器训练、试改限制、提示对照和检查的计算开销。

4.1 评估设置

三元组相似度使用主体、关系和值的字符对计数。三部分余弦相似度的权重为 0.4、0.2、0.4。相似度达到 0.92 的三元组对组成相似组，用于唯一性评分。层级检查使用管理、监督和成员关系的方向，跳过含义不明确的关系。实体检查作用于标记为实体引用的端点，其他字段值按文本处理。

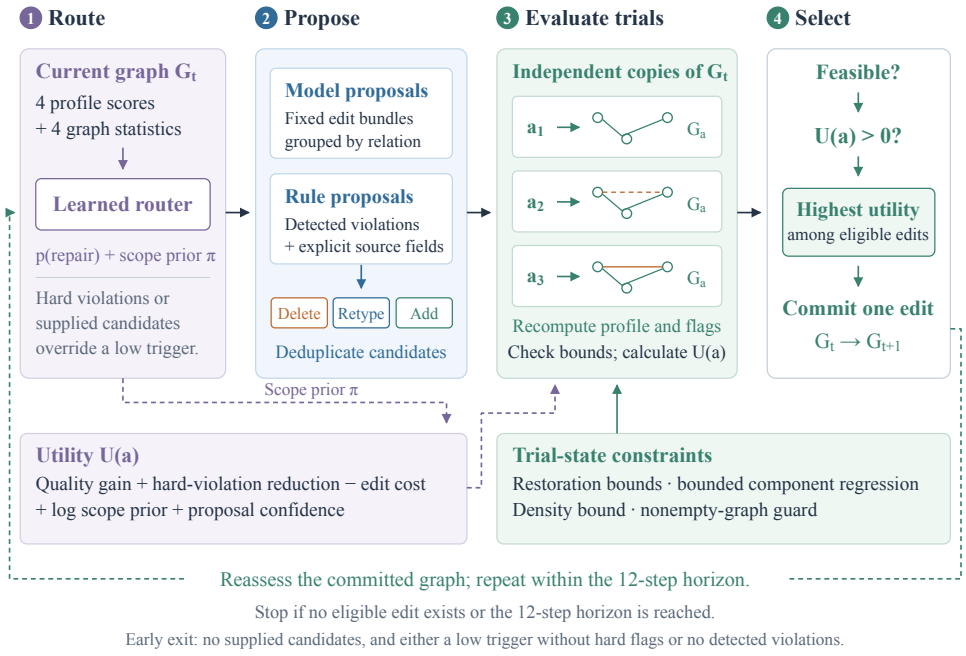


图 3 一轮试改检查。路由器为编辑范围赋权，选择器在图副本上为每项模型或规则编辑打分、检查边界，并执行最佳允许正分编辑。随后重新构建评估概况。保存建议测试最多允许十二轮。

空图的节点连接分为零，其余三项为 100。非空图没有源文本时，来源分为 50。选择器的来源检查去掉两个字符串中的空白，早期选择器日志保留相应运行使用的空白规则。

缺失字段检查读取文档、行或句子开头的允许标签之后的值。当原文给出一个字段而图中缺少它时，增加一个候选。重复标签给出冲突值时，跳过该字段。这项检查从来源中的明确字段提供候选。

4.2 路由器与训练设置

网络层大小为 $8 \to 32 \to 16$，具有一个修复输出和三个范围输出，共 884 个参数。两个隐藏层均使用 ReLU。sigmoid 给出修复分数，softmax 给出范围权重。训练和使用时共用特征代码。特征版本不匹配时，路由器改用等权重。

训练标签来自已知注入缺陷。损坏图的修复标签 y_j 为一，其干净配对图的标签为零。范围标签 c_j 取图中最常见的缺陷范围。干净图只训练修复输出。损失为

$$\mathcal{L}(\phi) = \frac{1}{N}\sum_j \mathrm{BCE}(p_j, y_j) + \frac{\sum_j b_j(-\log \pi_{j,c_j})}{\sum_j b_j + \epsilon}. \tag{15}$$

算法 2 带试改检查的尺度感知编辑选择

输入：　输入图 G、原文 X、已提供候选 A_0、任务输入、步数上限 T
输出：　修复图 $\widehat{G}$ 和决策日志 J

$\widehat{G} \leftarrow G$; $J \leftarrow \emptyset$; $E \leftarrow \emptyset$
将缺失来源字段的候选加入 A_0
对 $t = 1$ 到 T 执行
    为 $\widehat{G}$ 计算评估概况和 $(p_{\text{repair}}, \boldsymbol{\pi})$
    若 $A_0 = \emptyset$、$h(\widehat{G}) = 0$ 且 $p_{\text{repair}} < \tau_{\text{repair}}$ 则
        退出循环
    结束 若
    若 没有报告的违规且 $A_0 = \emptyset$ 则
        退出循环
    结束 若
    $A \leftarrow \text{Unique}(A_0 \cup \text{RuleEdits}(\widehat{G}))$
    跳过已在 E 中的图与候选组合
    对其余每个 $a \in A$ 在副本上试改，并将组合记入 E
    为每次试改检查图边界并计算 $U(a)$
    将试改分数和决策加入 J
    若 没有允许候选满足 $U(a) > 0$ 则
        退出循环
    结束 若
    对 $\widehat{G}$ 执行 $U(a)$ 最高的允许编辑
结束 对
返回 $\widehat{G}, J$

有范围标签时，b_j 为一；ϵ 避免除零。第一项训练修复分数，第二项用损坏图训练范围权重。同一文档组只属于训练、验证或测试中的一个划分。选择器根据任务输入和源文本计算特征。

4.3 候选与试改设置

在保存建议测试中，清理后输入与模型输出之间的差异按关系分组，每组是一个置信度为 0.7 的编辑包。学习先验测试将路由器与始终开启的等权先验比较。

四项分数的下界依次为 55、55、60、45。在满足上述边界规则的条件下，每项检查分数最多下降两分。试改后密度最多为 0.75。保存建议测试使用 $T = 12$ 和 $\tau_{\text{repair}} = 0.05$。

第二种评分从默认分数中减去 $0.05 \log(1/3 + 10^{-6})$，使等权重的先验项为零。对于固定路由器，它保留编辑排序，但改变正分接受门槛。两种评分均在保存建议测试前固定。

4.4 提示与规则设置

4.4.1 模型输入与输出解析

运行程序将原文、文档编号、允许关系和输入三元组传给模型。评分器加载参考值。测试通过改变参考值和缺陷标签，确认模型输入保持一致。旧加载器从参考记录中读取文档编号，作为任务信息。

JSON 解析器将无效响应记为空图，这些空图也计入评分。基础提示使用与错误报告提示相同的输入和输出上限，但去掉报告。

4.4.2 共享提示与响应

同一个系统提示支持基础输入、错误报告和 SHACL 检查三种设置。每个输入调用三次，得到三个响应。每个响应在过滤前后分别计分，共得到六个分数。每个过滤前后对照使用相同三元组。这样，在衡量过滤器时，候选生成保持固定。

模型为 `google/gemma-4-26B-A4B-it`，温度为零。每种设置的输出上限均为 4,000 个 token。提示长度不同，因此我们报告输入 token 数。请求时间包含重试和请求间隔等待。所有响应均保存，无效响应也计分。较早的 Claude 测试为每种方法使用独立提示。

4.4.3 SHACL 提示基线

Lin 等人使用五部分提示：引言、检测到的错误、SHACL 规则、图上下文和指令 (Lin et al. 2025)。我们将其完整规则与完整图（$M + G$）设置用于字段任务。pySHACL 工具检查文档编号、允许关系、非空值和字段数量。SPARQL 规则根据原文检查取值。

提示包含完整原文，并返回完整 JSON 图。每篇文档的错误合并到一次调用中。我们将该基线称为*Lin 风格 SHACL 上下文（改编版）*。RDF 保存唯一三元组，共享 JSON 输入保留重复副本。每种提示设置的输出都在过滤前后计分。

4.5 计算开销

采用哈希查找时，清理平均需要 $O(|G|)$ 次三元组操作。过滤 k 个候选的时间为 $O(k(1 + c_X))$，额外内存为 $O(k + |\mathcal{R}_X|)$。这里，c_X 表示在文本中查找一个值的成本，文本长度会影响这一成本。模型调用耗时与这些检查分别统计。

对于 m 条三元组，两两检查的成本为 $O(m^2)$。T 轮、每轮最多 K 次试改的成本为 $O(TKm^2)$，另加文本检查成本。主过滤器只扫描候选列表一次。

5 实验评价

实验围绕方法中的修复决策展开。我们先在注入缺陷和模型构建图上比较完整方法，再在共享条件下测试错误报告、过滤和编辑选择。收据与合同测试考察从源文本中选择字段值，人工评价检查错误和编辑的含义。本文提出三个问题。**RQ1**：修复何时能改善图？**RQ2**：上下文、过滤和编辑选择各改变了什么？**RQ3**：已有图何时比重新抽取更有用？表 12 汇总这些发现。

表 1 注入缺陷测试数据。最后三列为文档数。

领域	文档数	缺陷数	训练	验证	测试
政务	499	998	349	75	75
金融	500	1,000	350	75	75
环境	500	1,000	350	75	75
总计	1,499	2,998	1,049	225	225

5.1 测试设置

来源记录与注入缺陷。

来源数据包含陕西省人民政府的 4,728 条政务记录、国家金融监督管理总局的 1,000 条金融记录，以及生态环境部的 500 条环境记录。我们从每个来源最多抽取 500 篇文档，为每篇构建字段图。

源文本以“字段名：值”列出非空字段，用中文句号连接。每个值最多 320 个字符，服务标签最多 100 个字符。相同的清理后取值构成参考三元组。我们用随机种子 42，按 70/15/15 将文档划分为训练、验证和测试集。在加入缺陷前按来源编号划分，同一原始文档及其修改副本属于同一组。

每张测试图恰好加入两个缺陷。六种缺陷为三元组缺失、三元组重复、非法关系、反向边、无来源支持的值和层级冲突。保存的数据列出每条干净和修改后的三元组。测试集有 225 篇文档，每个来源 75 篇，共 450 个缺陷（表 1）。

模型构建的图。

Claude Haiku 为相同的 225 篇测试文档分别构建新图。输入为源文本、文档编号和允许关系。评分器将结构化字段作为参考值。十个响应的 JSON 无效，按空图计分，解析成功率为 95.56%。共有 77 张图与参考不同，包含 302 项三元组差异。两名评审检查其中固定抽样的 200 项差异，第三名评审解决分歧（第 5.5 节）。

修复方法。

所有方法获得相同输入图、源文本、文档编号和允许关系。

- **来源字段复制**：读取字段标签，直接从文本复制值。
- **不修复**：返回输入。
- **仅规则**：去重，将指向文档节点的边反向，删除非法头节点和关系。
- **SHACL 风格修复**：使用 SHACL 形状约束检查文档头节点和允许关系 (Knublauch and Kontokostas 2017)。
- **直接调用模型**：调用一次模型，根据原文返回完整图。
- **ReAct 风格智能体**：一次调用查找错误，另一次调用执行修复(Yao et al. 2023)。
- **简单流程**：清理输入，调用一次模型，并对响应去重。
- **诊断+门控**：在提示中增加错误报告，按文档编号、关系、来源匹配、重复数量和每字段值数量过滤输出。

除另行指定模型外，模型方法均在同一端点使用 `claude-haiku-4-5-20251001`(Anthropic 2025)，温度为零。所有分数均包含失败调用。注入

缺陷测试中，直接调用模型 和 简单流程 各有一个无效 JSON 响应，诊断＋门控 和 ReAct 均为零。

评分与统计检验。
当输出恢复各条干净三元组的数量，并去掉其修改形式时，该缺陷视为修复。我们还衡量正确事实保留、过度修复、三元组 F1、整图精确匹配、调用次数和请求时间。整图精确匹配同时检查三元组与重复数量。

令 G_0 为模型构建的输入图，G_1 为修复图，G^* 为参考图。令 d 统计多余和缺失的三元组，包括重复副本。参考差异减少率为

$$1 - \frac{d(G_1, G^*)}{d(G_0, G^*)},$$

该指标在 77 个与参考不同的输入上取平均。负值表示输出差异变多。事实保留和过度修复也按参考三元组计算。人工分数衡量评审是否发现错误、是否接受编辑。

置信区间（CI）使用 10,000 次配对 bootstrap 抽样。匹配测试每次抽取完整文档，使同一文档的两个注入缺陷保持在一起。配对符号随机化检验使用 10,000 次抽样和种子 42。较早的缺陷与整图匹配比较使用精确 McNemar 检验。模型输出差异数还使用单侧配对 Wilcoxon 检验。

首次匹配上下文测试使用未校正的 p 值。后续组件、收据上下文和保存响应测试按各表所列分组进行 Holm 校正。每个 95% 置信区间对应一个比较，Holm 校正作用于同一检验组的 p 值。

对照设计。
完整方法测试衡量全部修复步骤组合后的结果。提示测试固定模型、输入和输出上限，只改变上下文。过滤测试对同一批生成三元组在检查前后计分。选择器测试复用保存的建议，并改变编辑接受规则。这些对照分别衡量生成、过滤和动作选择阶段的作用。

5.2 整体修复效果

我们检验修复能否恢复缺失或错误事实，同时保留正确的输入事实。注入缺陷提供已知修复目标，模型构建图用于测试抽取留下的错误，双模型对照则检查完整方法使用不同生成模型时的表现。

5.2.1 注入缺陷修复

诊断＋门控 修复 98.00% 的缺陷，保留 99.32% 的干净事实，F1 达 99.17%（表 2）。简单流程 使用相同的一次调用上限，修复率为 95.11%，F1 为 96.34%。诊断＋门控 修复了 简单流程 漏掉的 15 个缺陷；简单流程 修复了 诊断＋门控 漏掉的两个缺陷（McNemar $p = 0.00235$）。配对修复率增益为 2.89 个百分点（95% 置信区间：1.11–4.89；$p = 0.0067$），F1 增益为 2.84 个百分点（95% 置信区间：1.65–4.25）。直接调用模型 的修复率为 92.44%。来源字段复制 直接读取记录中的字段名和值，不调用模型，修复率和整图匹配均为 100%。

表 2 225 篇测试文档、450 个注入缺陷上的修复结果。过度修复、调用数和请求时间越低越好。

方法	缺陷修复	事实保留	过度修复	三元组 F1	整图匹配	调用／篇	耗时／篇
来源字段复制	**1.0000**	**1.0000**	**0.0000**	**1.0000**	**1.0000**	0.00	<0.001 s
不修复	0.0000	**1.0000**	**0.0000**	0.7615	0.0000	0.00	<0.001 s
仅规则	0.3200	**1.0000**	**0.0000**	0.8472	0.0622	0.00	<0.001 s
SHACL 风格修复	0.0000	**1.0000**	**0.0000**	0.7998	0.0000	0.00	<0.001 s
直接调用模型	0.9244	0.9687	0.0758	0.9520	0.8267	1.00	7.25 s
ReAct 风格智能体	0.8911	0.9324	0.1667	0.9213	0.5956	2.00	20.50 s
简单流程	0.9511	0.9723	0.0705	0.9634	0.8400	1.00	9.27 s
诊断＋门控	0.9800	0.9932	0.0167	0.9917	0.9333	1.00	11.44 s

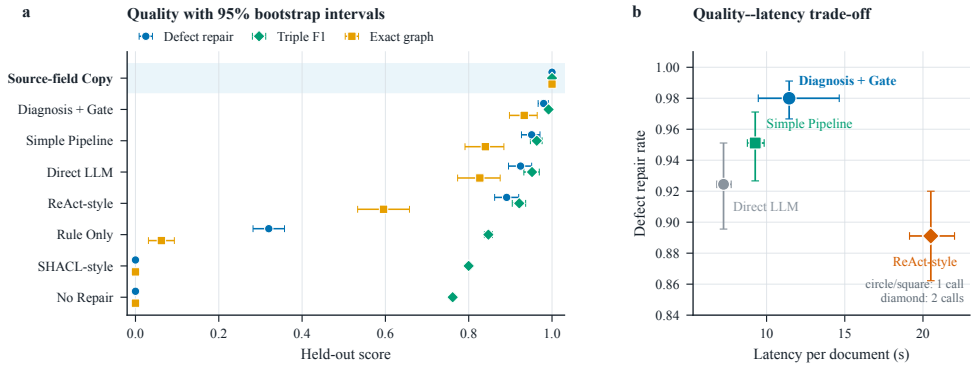


图 4 注入缺陷修复。**a**：修复率、三元组 F1 和整图匹配，带基于文档抽样的 95% 置信区间。**b**：一次或两次调用方法的分数与请求时间。

诊断＋门控 修复了 直接调用模型 漏掉的 28 个缺陷；直接调用模型 修复了 诊断＋门控 漏掉的三个缺陷（$p = 4.65 \times 10^{-6}$）。与 ReAct 比较时，两项计数分别为 42 和二（$p = 1.13 \times 10^{-10}$）。这些检验比较完整修复方法，第 5.3 节再测试其中步骤。

在相同调用上限下，组合流程相对 简单流程 同时提高了恢复效果和事实保留率。直接复制的结果也说明了来源格式的作用：这些记录将字段名和值并列写出，解析器可以直接恢复目标图。因此，这项测试清楚地检验了对已知修改的恢复能力。收据与合同测试进一步考察较不规则文本中的字段选择。

5.2.2 模型构建图的修复

接下来，我们使用相同来源记录和参考值，修复抽取模型生成的图。这项测试考察模型错误的恢复，以及原本正确字段的保留。

输入图的 F1 为 87.99%，整图匹配为 65.78%（表 3）。诊断＋门控 将 F1 提高到 92.48%，参考差异减少 27.75%，并保留全部原本匹配参考的输入三元组。简单流程 在 77 个有差异输入上的平均差异减少率为 -59.15%，增加的差异多于去掉的差异。

诊断＋门控 的 F1 比 简单流程 高 4.40 个百分点（95% 置信区间：3.07–5.77），平均每篇少 1.03 项差异（95% 置信区间：0.77–1.32；Wilcoxon $p = 3.18 \times 10^{-12}$）。三个图在 诊断＋门控 下完全匹配参考，在 简单流程 下不匹配；反向情况为零（$p = 0.25$）。来源字段复制 匹配全部参考。

表 3 模型构建图的修复。差异减少率使用 77 个与参考不同的输入，其余分数使用全部 225 篇文档。

方法	解析率	非精确输入	差异减少率	事实保留	过度修复	三元组 F1	整图匹配
来源字段复制	1.0000	77	1.0000	1.0000	0.0000	1.0000	1.0000
抽取图，不修复	0.9556	77	0.0000	1.0000	0.0000	0.8799	0.6578
简单流程	1.0000	77	−0.5915	0.9971	0.2389	0.8807	0.6711
诊断＋门控	0.9956	77	0.2775	1.0000	0.0292	0.9248	0.6844

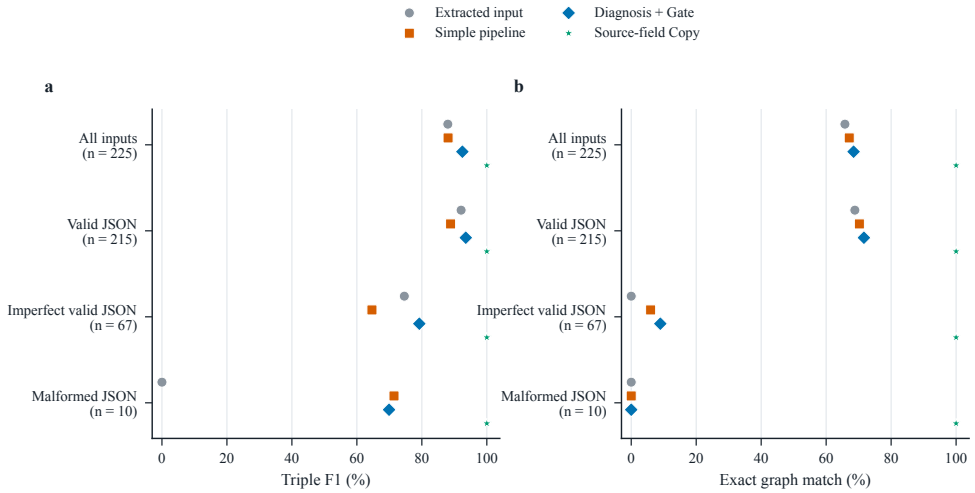


图 5 按输入类型划分的修复分数。有效 JSON 组展示可解析图的修复，无效 JSON 组展示空输入的恢复。

有效与无效 JSON 输入。

215 个有效 JSON 输入的 F1 为 92.09%。诊断＋门控 达到 93.53%，简单流程 为 88.85%。67 个有效但有参考差异的输入中，诊断＋门控 使差异减少 24.00%，F1 从 74.61% 提高到 79.22%。148 个原本精确匹配的图全部保持精确匹配。十个无效输入修复后的 F1 为 69.91%，没有一个整图精确匹配。

有效输入的结果展示了已解析图内部的修复，无效输入组则衡量从空图中恢复的情况。所有初始完全匹配的图继续完全匹配，表明主流程在这项测试中保留了这些完整输入。F1 的变化大于整图匹配率的变化，说明收益主要体现在字段层面：修正部分字段可以改善图，而其他差异仍然存在。

5.2.3 双模型测试

我们在 60 个注入缺陷输入上比较 直接调用模型、简单流程 和 诊断＋门控，每个来源 20 个。模型为 Claude Haiku 和 `google/gemma-4-26B-A4B-it`(Gemma Team 2026)。每种方法跨模型使用相同输入、提示形式、温度和一次调用上限。诊断＋门控 在两个模型上的修复率均最高（表 4）。它相对 简单流程 的 F1 优势在 Claude 上更大。Gemma 直接输出的 F1 高于 Claude。

这项对照中，组合流程在两个模型上均取得最高修复率，但 F1 增益的大小随生成模型变化。这将完整流程的结果与其接收的候选联系起来；下面通过共享提示和保存响应的测试考察各个步骤。

表 4 两个模型在相同 60 个注入缺陷输入上的结果。

模型	方法	修复率	F1	精确匹配
Claude	直接	0.958	0.970	0.867
Claude	简单流程	0.958	0.966	0.833
Claude	**诊断＋门控**	**0.975**	**0.991**	**0.917**
Gemma	直接	0.958	0.988	0.883
Gemma	简单流程	0.967	0.984	0.867
Gemma	**诊断＋门控**	**0.983**	**0.990**	**0.917**

表 5 Gemma 提示测试，每组输入 60 篇文档。每个原始／过滤对照使用同一响应。

上下文	输出	受控修复	受控 F1	模型构建 F1	模型构建整图匹配
基础提示	未过滤	0.9750	0.9951	0.9185	0.6667
基础提示	门控	0.9750	0.9962	0.9252	0.6667
诊断提示	未过滤	0.9250	0.9850	0.9079	0.6667
诊断提示	门控	0.9250	0.9861	0.9147	0.6667
SHACL 上下文	未过滤	0.9500	0.9912	0.9322	0.7000
SHACL 上下文	门控	0.9500	0.9912	0.9364	0.7000

5.3 修复组件的作用

下面考察评估报告、输出约束和编辑选择器如何影响修复。对照按各组件作用的阶段组织：报告改变提示，门控改变生成输出，选择器决定哪些候选编辑进入图。

5.3.1 错误报告与提示上下文

Gemma 测试使用 60 篇文档，每个来源 20 篇，分别测试注入缺陷输入和模型构建输入。三个提示共享指令、清理步骤、温度和输出上限。每个响应在同一门控过滤前后计分。表 5 给出六种设置，表 20 给出输入 token 数和请求时间。第 4.4.3 节给出 SHACL 提示。

经过过滤，基础提示在注入缺陷输入上达到 99.62% F1 和 97.50% 修复率。错误报告提示为 98.61% F1 和 92.50% 修复率。配对 F1 变化为 -1.01 个百分点（95% 置信区间：$[-1.67, -0.38]$；$p = 0.0082$）。模型构建输入上，基础提示 + 门控 的 F1 为 92.52%，诊断＋门控 为 91.47%，SHACL 上下文＋门控 为 93.64%。诊断 与 SHACL 的 F1 差为 -2.16 个百分点（95% 置信区间：$[-5.26, 0.02]$；$p = 0.105$）。

在两个输入组中，错误报告提示的平均 F1 均低于基础提示。报告中的问题列表改变了模型建议，而基础提示在这项对照中获得了更好的字段匹配。SHACL 上下文在模型构建输入上取得最高平均 F1。因此，提示中的内容与输出检查共同影响结果。

5.3.2 过滤同一批生成三元组

在保存的 Claude 响应上，过滤将 F1 从 98.95% 提高到 99.17%，过度修复降低 0.60 个百分点。修复率保持 98.00%，整图匹配保持 93.33%。过滤器去掉八条三元组，均未通过来源检查且与参考不同。附录 A.1 给出完整方法表。

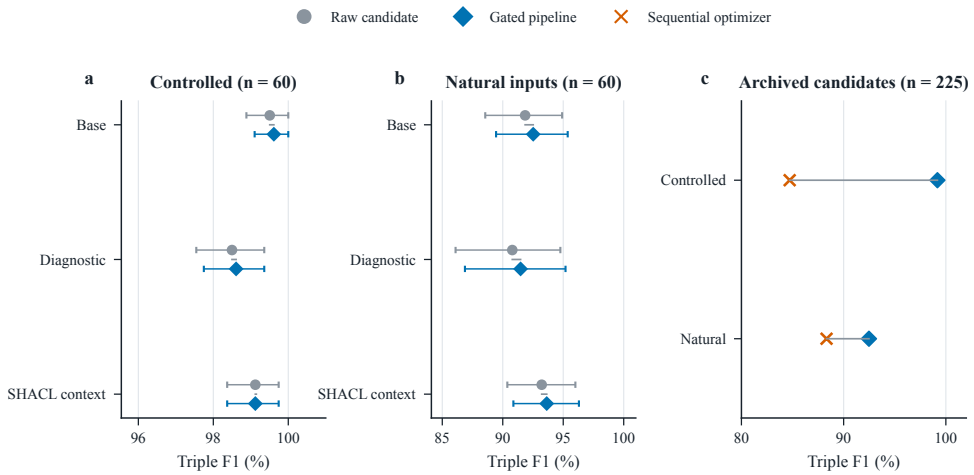


图 6 提示、过滤和选择器测试。a–b 使用 60 篇文档的 Gemma 输出；每个原始／过滤对照使用同一响应，误差线表示 95% 文档 bootstrap 置信区间。c 使用 225 篇文档保存的 Claude 建议。

在使用错误报告提示的模型构建输入上，过滤将 F1 从 90.79% 提高到 91.47%，增益为 0.69 个百分点（95% 置信区间：0.20–1.29；$p = 0.0335$），整图匹配保持相同。360 个响应共拒绝 27 条三元组，均与参考不同。

各项过滤检查。

我们将 540 个已保存响应经过八种过滤设置，得到 4,320 个输出。完整过滤器拒绝 28 条三元组，其中 11 条文档编号错误，17 条来源匹配失败，均与参考不同。其他检查在该样本中未额外去掉候选。21 项检查比较的校正后 p 值均大于 0.05。附录 A.2 给出结果表。

这些测试中，被拒绝的三元组来自来源和文档编号检查。门控删除不满足这些要求的候选，从而提高字段级分数。在保存的 Claude 响应对照中，修复率和整图匹配率保持不变。为字段选择正确的来源值，仍发生在生成和候选排序阶段。

5.3.3 清理、报告与过滤的组合

我们分别开启或关闭输入清理（P）与错误报告（D）。四种提示设置使用 60 个注入缺陷输入和 60 个模型构建输入，得到 480 个 Gemma 响应。每个响应分别在启用和关闭门控（G）时计分。每项主效应都对另外两个开关取平均。

六项 Holm 校正后的 p 值均大于 0.05（表 6）。错误报告在两组输入中都降低平均 F1。门控在注入缺陷输入上提高 0.164 个百分点，在模型构建输入上提高 0.707 个百分点。所有输入组共拒绝 47 条三元组，其中 17 条文档编号错误，30 条未通过来源匹配，均与参考不同。公开分数还列出开关联合效应和每项检查去掉的候选。

对其他开关取平均后，门控在两个组中的平均效应均为正，错误报告的平均效应均为负。匹配上下文对照也呈现相同方向。拒绝日志通过具体的来源与编号检查失败解释门控产生的变化；报告则在更早的生成阶段发挥作用。

表 6 平均 F1 变化，单位为百分点。每组输入有 60 篇配对文档，各效应对另外两个开关取平均。区间为 95% 文档 bootstrap 置信区间。p_H 对六项主效应检验进行 Holm 校正。

输入	组件	变化	95% 区间	p_H
受控输入	预处理	-0.028	[-0.797, 0.778]	1.000
受控输入	诊断	-0.651	[-1.348, -0.056]	0.164
受控输入	门控	+0.164	[0.054, 0.301]	0.156
模型构建输入	预处理	+0.697	[0.000, 1.905]	1.000
模型构建输入	诊断	-0.535	[-2.376, 0.842]	1.000
模型构建输入	门控	+0.707	[0.225, 1.316]	0.086

5.3.4 图边界下的编辑选择

选择器测试考察质量分数、编辑成本和范围权重能否从保存的建议中选出有用修改。在保存的 Claude 建议上，默认选择器使用学习权重或等权重时，都未接受注入缺陷输入上的编辑。修复率保持为 32.00%，F1 为 84.72%，即清理后输入的分数。在模型构建输入上，当前选择器使用等权重时 F1 为 88.33%，使用学习权重时为 87.99%；学习触发器也为 87.99%。去掉编辑成本后，注入缺陷 F1 提高到 89.30%，该设置下的模型构建 F1 为 88.13%。

日志显示，一些图虽然质量分数较高，却仍缺少字段。部分有用的添加获得零分或负分，因而被拒绝。结合成本测试，这些结果指出接受评分是编辑选择中的关键环节：已有的修正候选需要获得足够增益，才能通过正分门槛。在模型构建输入上，等范围权重的分数高于学习权重。附录 A.2 给出全部 13 种设置、两种先验评分形式和 11,700 个结果；附录 A 还给出早期选择器结果和调用策略测试。

5.4 收据与合同修复

这些测试考察需要从文档文本中选择的字段。收据测试比较来源行提示、提示内容及已有图的作用，合同测试将相同修复思路应用于另一类文档任务。我们同时跟踪恢复的字段和修复时丢失的原正确匹配。

5.4.1 新收据上的来源行提示

我们使用 SROIE 收据(Huang et al. 2019)，检验字段行提示能否帮助模型从完整来源中选择取值。对照固定字段含义和源文本，只改变额外上下文。附录 A.3 给出初次收据测试和开发细节。

我们用 20 张新收据设置索引设计，然后固定设计，在另外 60 张收据上测试（第 3.3.2 节）。首次 60 张、开发 20 张和新测试 60 张收据的编号不同，空白清理后的文本也不同。抽样种子为 20260923。所有预测保存后才下载测试标签。每种方法得到相同字段含义、带行号原文、输入图和指令。简单流程 直接使用这些输入；证据索引 增加字段行提示；SHACL 增加规则检查上下文。三种方法的过滤器使用相同空白规则。

主要比较为 证据索引 相对 简单流程 的平均文档 F1 差值。代码和评分规则在测试前固定，表 7 给出输出分数。

表 7 60 张新收据的修复。各方法共享字段含义、来源行和空白检查。精确匹配使用有标签字段。

方法	三元组 F1	整图匹配	规范化 F1	事实保留
抽取输入	0.7833	0.3500	0.7958	1.0000
简单流程, 未过滤	0.7875	0.2833	0.8083	0.9250
简单流程, 过滤后	0.7875	0.2833	0.8083	0.9250
证据索引, 未过滤	0.8333	0.4000	0.8625	0.9306
证据索引, 过滤后	0.8351	0.4000	0.8643	0.9306
SHACL 上下文，未过滤	0.7833	0.2833	0.8042	0.9347
SHACL 上下文，过滤后	0.7833	0.2833	0.8042	0.9347

新测试集的输入 F1 为 78.33%。过滤后 简单流程、证据索引、SHACL 分别为 78.75%、83.51%、78.33%。简单流程 比输入高 0.42 个百分点（95% 置信区间：$[-4.17, 5.00]$）。证据索引 比 简单流程 高 4.76 个百分点（95% 置信区间：$[2.50, 7.26]$；配对随机化 $p = 0.0004$）。三种方法的整图匹配分别为 28.33%、40.00%、28.33%。

输入有 52 个字段与参考不同。简单流程 恢复其中 16 个，并丢失 15 个原本精确匹配的字段。证据索引 恢复 26 个、丢失 14 个；SHACL 恢复 13 个、丢失 13 个。索引修正了 11 项标点差异和 6 项文本边界差异。它丢失的 14 个精确匹配中，有 10 个数值金额相同，只改变显示格式。

金额检查去掉总额字段中的货币前缀和千位分隔符。正确金额数为：输入 58/60，简单流程 59/60，证据索引 59/60，SHACL 58/60。清理大小写和空白后，证据索引 的 F1 为 86.43%，简单流程 为 80.83%。索引同时提高精确和清理后文本分数。

过滤器拒绝 1 个候选。它与参考不同，并将地址片段拼成原文中没有的文本。被拒绝的参考匹配候选数为 0。过滤使索引 F1 提高 0.18 个百分点，另外两种方法的 F1 保持不变。

索引提示比 简单流程 恢复了更多参考字段，丢失的原精确匹配也更少。金额检查还解释了为什么部分丢失的精确匹配仍保留相同数值。F1 差异的大部分在过滤前就已出现，说明主要收益位于生成阶段。去掉大小写和空白差异后仍有改善，也表明收益超出了这两类格式差异。

5.4.2 来源行提示对照

接下来，我们测试索引提示中的哪些部分影响字段选择。六种提示设置使用相同的 60 张新测试收据。这一独立批次包含与组件测试一起收集的 360 个新 Gemma 响应。所有响应都能解析。各设置均保留完整来源，并共享模型、输出上限和过滤器。随机索引与完整索引的字符长度接近。

完整索引的 F1 比 简单流程 高 4.35 个百分点（95% 置信区间：1.25 至 7.86）。五项预设比较的 Holm 校正值为 $p_H = 0.065$。完整索引与仅锚点行的平均 F1 相同。完整索引与随机索引相差 1.85 个百分点（置信区间：$[-0.65, 4.52]$；$p_H = 0.696$）。去掉字段含义使完整索引 F1 下降 8.81 个百分点（置信区间：$[4.23, 13.39]$；$p_H = 0.006$）。本批次 简单流程 去掉字段含义后得分更高。这些结果说明字段含义有助于索引提示选择取值。

在这一批次中，仅锚点行与完整行窗口取得相同平均分数；完整索引与随机索引的差值区间同时包含正值和负值。字段含义在索引提示中发挥了明确作用，而在 简

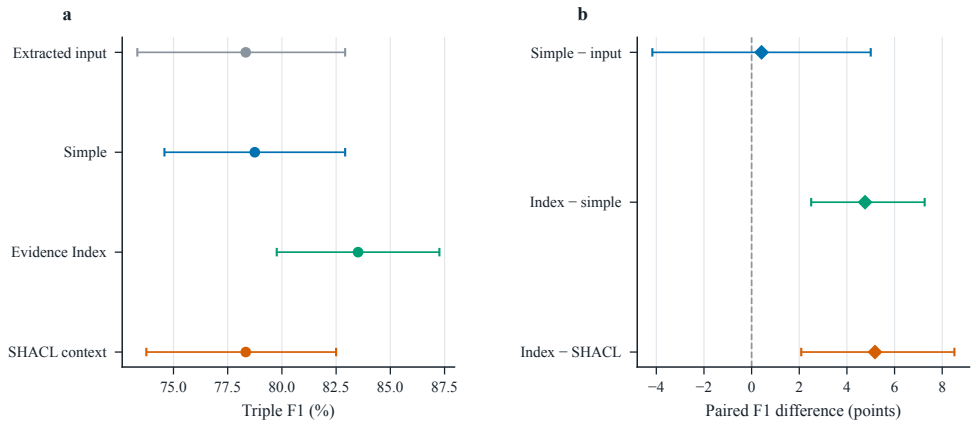


图 7 新收据测试。左：平均 F1 及 95% 文档 bootstrap 置信区间。右：配对 F1 变化。所有方法共享字段含义和来源行。

表 8 60 张收据上的来源行测试。各行使用相同模型、原文、输出上限和过滤器。随机提示与索引的字符长度大致相同。

上下文	精确 F1	规范化 F1
简单流程	0.7917	0.8125
完整索引	0.8351	0.8643
仅锚点	0.8351	0.8643
随机索引	0.8167	0.8458
简单流程，无字段定义	0.8083	0.8292
完整索引，无字段定义	0.7470	0.7762

单流程 中的影响方向相反。因此，上下文的作用取决于字段说明与行提示如何共同呈现。

5.4.3 修复与重新抽取

为测试已有图的作用，我们在 60 张新测试收据上使用四种设置：带或不带索引的修复与重新抽取。各设置共享来源行、字段含义、文档编号和输出上限，修复还接收已有图。全部 240 个响应都将 JSON 放在 Markdown 围栏中，固定的严格解析器将它们计为失败，因此每种设置的 F1 均为零。

在收集后的单独分析中，我们去掉围栏，并为每个输出赋予已知文档编号，对这些保存响应重新计分。索引修复的 F1 为 83.93%，索引重新抽取为 74.86%。配对差值为 9.07 个百分点（95% 区间：5.22 至 13.10）。在这一评分规则下，已有图帮助生成更多匹配的字段值。建图成本与修复分开计算。附录 A.4 给出解析步骤、编号错误和来源行覆盖情况。

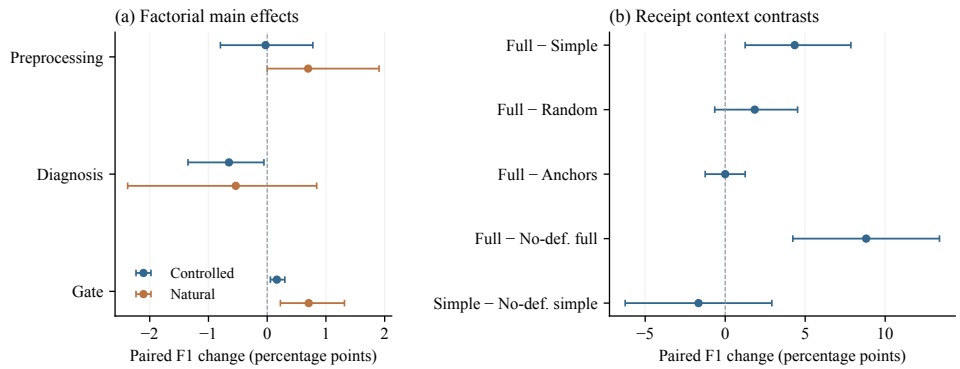


图 8 组件与来源行测试。点表示平均配对 F1 变化，误差线为各项变化的 95% 置信区间。组件测试与收据测试分别进行 Holm 校正。

表 9 56 份文档的合同结果。F1 检查空白清理后的精确文本。“丢失”统计被删除的正确输入事实。输入行给出初始图成本，修复行给出修复调用成本。

输出	F1 (%)	丢失	新增不匹配	请求数
初始抽取	46.76	0	0	59
简单流程修复	43.35	7	2	71
索引修复	44.84	5	12	69
索引重新抽取	44.12	8	32	71

5.4.4 合同字段测试

合同测试在另一项字段任务上考察相同修复思路，将修复同时与输入图和重新抽取比较，从而衡量修改已有图带来的收益和损失。

我们用 CUAD(Hendrycks et al. 2021) 测试五个字段：文档名称、签约日期、生效日期、到期日期或初始期限，以及适用法律。选择空白清理后每字段最多一个不同参考值、原文不超过 40,000 字符的合同。保留完整原文，并检查标注值的位置。20 份官方训练合同构成开发集；全部 56 份满足规则的官方测试合同构成测试集。

比较初始抽取、简单流程 修复、索引修复和索引重新抽取。两种修复方法得到相同初始图，索引方法在完整原文之外增加字段行窗口。各设置共享 Gemma 模型、字段含义和 4,000 token 输出上限。采集前，程序已设置 JSON 围栏处理和文档编号分配。一轮开发之后，提示与选择规则保持固定。

索引修复 F1 为 44.84%，输入为 46.76%，下降 1.92 个百分点。它删除五条正确输入事实，并增加十二条与参考不同的事实。简单流程 修复也低于输入。

索引修复的 F1 比索引重新抽取高 0.72 个百分点（95% 配对置信区间：[-4.80,6.48]；符号翻转检验 $p = 0.810$）。测试共用 270 次请求得到 224 个输出，包括重试。失败输出计为空图。评分在空白清理后，将复制文本与 CUAD 标签比较。

输入图在这些设置中取得最高 F1。两种修复方法都丢失了正确输入事实，索引修复还增加了与参考不同的值。它相对索引重新抽取的小幅领先，其区间同时覆盖两个

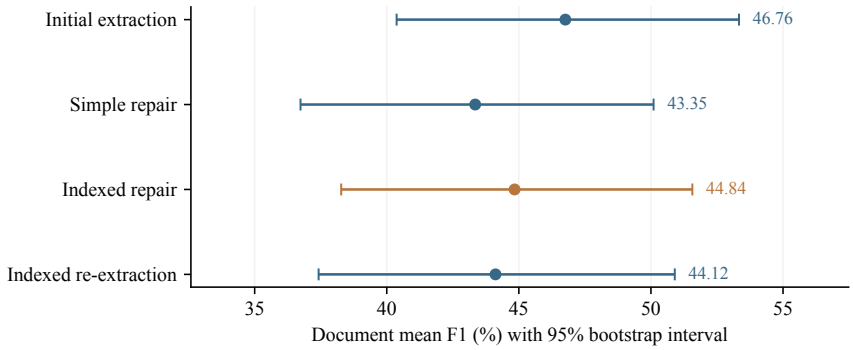


图 9 合同分数。点表示文档平均 F1，误差线为各方法的 95% 置信区间。正文给出修复与抽取的配对差值区间。

表 10 人工评分。一致率和 κ 使用两位评审原始的 0/1/U 答案。正例数使用最终标签，接受率使用答案为 0 或 1 的项目。

任务	判断	一致率	κ	正例／确定项	U
D	输入错误	80.0%	0.600	95/200 (47.5%)	0
D	建议编辑可接受	62.0%	0.334	86/200 (43.0%)	0
E	输入错误	86.0%	0.677	134/200 (67.0%)	0
E	实际编辑可接受	65.0%	0.128	134/199 (67.3%)	1

方向。收据上的收益在这项合同任务上变成了相对输入的下降。这些结果指出，保留正确字段和选择与标注一致的文本片段，是主要修复目标。

5.5 错误与编辑的人工评价

人工评价考察自动评分背后的两项判断：参考差异在上下文中是否属于错误，以及某项编辑在输出图中是否可接受。前者针对输入事实，后者还考虑操作对图的影响。

任务 D 从模型构建输入的 302 项三元组差异中抽取 200 项参考差异。任务 E 从五种设置的 598 项编辑中抽取 200 项实际编辑。两者都用种子 42 随机抽样。每项是一项差异或编辑，同一文档可提供多项。评审看到原文、模式、输入图和指定操作。任务 E 还展示完整输出图。方法名称与参考质量提示隐藏。

两名具有计算机背景和中文阅读能力的评审先阅读规则和练习样例，再各自独立标注相同的 400 项。每项回答两个问题：输入在该位置是否有错（`is_error`），以及添加或删除操作是否可接受（`repair_acceptable`）。标签为 0、1 或 U，并说明理由。U 表示所示证据不足以作出判断。第三名评审检查原文和图，在两人的答案中选择，解决 154 项分歧。其余 246 项保留一致标签。一项编辑接受判断仍为 U。原始标签和一致性分数均保留。

评审在 95/200 项参考差异中发现输入错误，接受 86/200 项建议编辑，以及 134/199 项实际编辑，另有一项不确定。一个缺失值本身可以是正确的，但添加后可能使仅允许一个值的字段出现两个值。输入错误与编辑接受问题分别覆盖这两个判断。任务 E 的原始编辑一致率为 65.0%，$\kappa = 0.128$。

表 11 按设置划分的实际编辑接受率。随机样本的各组大小不同，比例使用有明确判断的编辑。

配置	抽样数	可接受	U
历史简单流程	105	63/104 (60.6%)	1
历史诊断＋门控	41	25/41 (61.0%)	0
Gemma 基础提示＋门控	16	13/16 (81.3%)	0
Gemma 诊断＋门控	16	13/16 (81.3%)	0
Gemma SHACL 上下文＋门控	22	20/22 (90.9%)	0

表 11 报告各设置的样本量和可接受编辑数，合并接受率汇总这些组。两个人工问题得到不同结果，因为即使值本身正确，添加方式也可能违反字段规则。原始一致性分数表明，与识别输入错误相比，评审对编辑接受判断的分歧更大。标签和理由共同帮助定位字段选择与编辑执行中的问题。

5.6 讨论

来源格式与字段选择。

中文记录直接给出字段名和值，因此 来源字段复制 可以匹配参考。收据需要在文档中的多个值之间选择。在新收据测试中，索引提示改善了这一选择，收益在过滤前就已出现。上下文对照表明，字段含义与行提示共同发挥作用。合同则呈现不同结果：修复丢失了正确输入匹配，最终低于初始图。来源格式与保留正确输入字段的能力共同影响修复价值。

检查与字段含义。

输出门控删除未通过来源与文档编号检查的三元组。为关系选择正确的来源值，则是修复的另一项工作。收据中可能同时出现小计和最终金额，两者都在文本中。字段含义决定哪一个属于“总额”。人工评价在精确参考匹配之外考察这些上下文，并检查最终编辑是否符合图的字段规则。

评估与下一项编辑。

评估概况将检测到的问题连接到生成和动作选择。匹配测试中，错误报告组的平均 F1 更低；选择器日志则显示，一些有用编辑被接受评分拒绝。这些结果指出两个具体设计目标：报告应帮助模型选择字段值，试改评分应在质量和编辑成本之外计入缺失字段恢复的收益。表 12 汇总三个问题的答案。

5.7 代码与数据

代码、提示、数据名单、模型输出和评分脚本位于 https://github.com/turambar928/MCP_based_KG_construction。每次运行保存模型设置、解析状态、请求和 token 用量，全部响应均保留。各实验 README 给出数据来源、样本、校验值和运行命令。

人工评价文件包含固定任务规则、评分器、汇总结果、原始一致性分数和样本名单。个人答案与理由不公开。收据评分使用 SROIE 的人工字段标签。

表 12 研究问题与测试结果。

问题	比较	结果
RQ1	注入缺陷与模型构建图	诊断＋门控改善修复率和 F1。直接复制匹配全部中文记录参考。
RQ2	共享提示与响应；来源行提示；编辑选择器	来源和编号检查去掉多余三元组。收据索引提高 F1。匹配测试中，错误报告降低平均 F1。模型构建输入上，等权选择器得分高于学习权重。
RQ3	收据、合同修复与重新抽取	收据测试在围栏和编号清理后倾向已有图。合同索引修复得分低于初始图。

6 结论

我们将图检查与利用源文本和字段约束的修复结合。评估概况记录局部、图和来源错误。主路径先清理输入，再请求模型生成完整图，最后过滤三元组。尺度感知选择器还在图副本上比较编辑，考虑质量增益、成本和图边界。共享提示以及过滤前后的评分展示各步骤带来的变化。

诊断＋门控 修复了 98.00% 的注入缺陷，并将模型构建图的 F1 从 87.99% 提高到 92.48%。直接字段复制恢复了中文记录中的全部参考。来源行提示使收据 F1 比简单流程 高 4.76 个百分点。后续测试中，完整索引与仅锚点的平均 F1 相同。合同索引修复的 F1 为 44.84%，输入为 46.76%；与重新抽取相差 0.72 个百分点（95% 置信区间：$[-4.80, 6.48]$）。来源和文档编号检查去掉了不需要的三元组。匹配测试中，错误报告降低了平均 F1；模型构建输入上，默认学习选择器的分数低于等权重设置。

人工评价在 95/200 项参考差异中发现输入错误。评审接受五种设置中的 134/199 项编辑，另有一项不确定。这些结果给出清楚的改进目标：为每个字段选择正确取值，保留正确的输入事实，并检查最终图。

声明

数据可用性

数据名单、固定样本编号、模型输出和评分脚本位于 https://github.com/turambar928/MCP_based_KG_construction。各实验 README 给出来源数据集、访问链接和数据修改。我们公开人工评价总数和固定抽样规则。个人答案、书面理由和来源上下文不公开。

代码可用性

同一仓库包含代码、完整提示、模型设置，以及复现论文分析的运行命令。

参考文献

Acosta M, Zaveri A, Simperl E, et al (2013) Crowdsourcing linked data quality assessment. In: International Semantic Web Conference, pp 260–276, https://doi.org/10.1007/978-3-642-41338-4_17

Anthropic (2025) Introducing Claude Haiku 4.5. Anthropic model release announcement, URL https://www.anthropic.com/news/claude-haiku-4-5, published 15 October 2025; accessed 17 September 2026

Auer S, Bizer C, Kobilarov G, et al (2007) DBpedia: A nucleus for a web of open data. In: The Semantic Web: 6th International Semantic Web Conference and 2nd Asian Semantic Web Conference (ISWC/ASWC 2007). Springer, pp 722–735, https://doi.org/10.1007/978-3-540-76298-0_52

Bian H (2025) LLM-empowered knowledge graph construction: A survey. arXiv preprint, https://doi.org/10.48550/arXiv.2510.20345, arXiv:2510.20345

Bollacker K, Evans C, Paritosh P, et al (2008) Freebase: A collaboratively created graph database for structuring human knowledge. In: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, pp 1247–1250, https://doi.org/10.1145/1376616.1376746

Bordes A, Usunier N, Garcia-Duran A, et al (2013) Translating embeddings for modeling multi-relational data. In: Advances in Neural Information Processing Systems, URL https://papers.nips.cc/paper_files/paper/2013/hash/1cecc7a77928ca8133fa24680a88d2f9-Abstract.html

Chen M, Zhang W, Zhang W, et al (2019) Meta relational learning for few-shot link prediction in knowledge graphs. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp 4217–4226, https://doi.org/10.18653/v1/D19-1431

Das R, Dhuliawala S, Zaheer M, et al (2018) Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning. In: International Conference on Learning Representations, URL https://arxiv.org/abs/1711.05851

Dettmers T, Minervini P, Stenetorp P, et al (2018) Convolutional 2D knowledge graph embeddings. In: Proceedings of the AAAI Conference on Artificial Intelligence, https://doi.org/10.1609/aaai.v32i1.11573

Ding B, Wang Q, Wang B, et al (2018) Improving knowledge graph embedding using simple constraints. In: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, pp 110–121, https://doi.org/10.18653/v1/p18-1011

Dong X, Gabrilovich E, Heitz G, et al (2014) Knowledge vault: A web-scale approach to probabilistic knowledge fusion. In: Proceedings of the 20th

ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp 601–610, https://doi.org/10.1145/2623330.2623623

Galárraga LA, Teflioudi C, Hose K, et al (2013) AMIE: Association rule mining under incomplete evidence in ontological knowledge bases. In: Proceedings of the 22nd International Conference on World Wide Web, pp 413–422, https://doi.org/10.1145/2488388.2488425

Gemma Team (2026) Gemma 4 technical report. arXiv preprint, https://doi.org/10.48550/arXiv.2607.02770, arXiv:2607.02770

Guo S, Wang Q, Wang L, et al (2016) Jointly embedding knowledge graphs and logical rules. In: Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, pp 192–202, https://doi.org/10.18653/v1/d16-1019

Hendrycks D, Burns C, Chen A, et al (2021) CUAD: An expert-annotated NLP dataset for legal contract review. In: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, URL https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/hash/6ea9ab1baa0efb9e19094440c317e21b-Abstract-round1.html

Huang Z, Chen K, He J, et al (2019) ICDAR2019 competition on scanned receipt OCR and information extraction. In: 2019 International Conference on Document Analysis and Recognition (ICDAR). IEEE, pp 1516–1520, https://doi.org/10.1109/ICDAR.2019.00244

Ji S, Pan S, Cambria E, et al (2022) A survey on knowledge graphs: Representation, acquisition, and applications. IEEE Transactions on Neural Networks and Learning Systems 33(2):494–514. https://doi.org/10.1109/tnnls.2021.3070843

Knublauch H, Kontokostas D (2017) Shapes constraint language (SHACL). W3C Recommendation, URL https://www.w3.org/TR/2017/REC-shacl-20170720/

Kontokostas D, Westphal P, Auer S, et al (2014) Test-driven evaluation of linked data quality. In: Proceedings of the 23rd International Conference on World Wide Web, pp 747–758, https://doi.org/10.1145/2566486.2568002

Lao N, Mitchell T, Cohen WW (2011) Random walk inference and learning in a large scale knowledge base. In: Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing, pp 529–539, URL https://aclanthology.org/D11-1049/

Lewis P, Perez E, Piktus A, et al (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems, pp 9459–9474, URL https://papers.nips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

Lin TW, Fierro G, Li H, et al (2025) Systematic evaluation of knowledge graph repair with large language models. arXiv preprint, https://doi.org/10.48550/arXiv.2507.22419, arXiv:2507.22419

Neo4j Labs (2024) LLM Knowledge Graph Builder. Open-source software repository, URL https://github.com/neo4j-labs/llm-graph-builder, version inspected at commit 5ff7af3e9bb9226e1bbecd02f70f8d98697727a7; accessed 16 September 2026

Nickel M, Murphy K, Tresp V, et al (2016) A review of relational machine learning for knowledge graphs. Proceedings of the IEEE 104(1):11–33. https://doi.org/10.1109/jproc.2015.2483592

Ortona S, Meduri VV, Papotti P (2018) Robust discovery of positive and negative rules in knowledge bases. In: Proceedings of the 34th IEEE International Conference on Data Engineering, pp 1168–1179, https://doi.org/10.1109/icde.2018.00108

Pan JZ, Razniewski S, Kalo JC, et al (2023) Large language models and knowledge graphs: Opportunities and challenges. arXiv preprint, https://doi.org/10.48550/arXiv.2308.06374, arXiv:2308.06374

Pan S, Luo L, Wang Y, et al (2024) Unifying large language models and knowledge graphs: A roadmap. IEEE Transactions on Knowledge and Data Engineering 36(7):3580–3599. https://doi.org/10.1109/tkde.2024.3352100

Paulheim H (2017) Knowledge graph refinement: A survey of approaches and evaluation methods. Semantic Web 8(3):489–508. https://doi.org/10.3233/sw-160218

Paulheim H, Bizer C (2014) Improving the quality of linked data using statistical distributions. International Journal on Semantic Web and Information Systems 10(2):63–86. https://doi.org/10.4018/ijswis.2014040104

Qu M, Tang J (2019) Probabilistic logic neural networks for reasoning. In: Advances in Neural Information Processing Systems, URL https://papers.nips.cc/paper_files/paper/2019/hash/13e5ebb0fa112fe1b31a1067962d74a7-Abstract.html

Qu M, Chen J, Xhonneux LP, et al (2021) RNNLogic: Learning logic rules for reasoning on knowledge graphs. In: International Conference on Learning Representations, URL https://arxiv.org/abs/2010.04029

Schlichtkrull M, Kipf TN, Bloem P, et al (2018) Modeling relational data with graph convolutional networks. In: European Semantic Web Conference, pp 593–607, https://doi.org/10.1007/978-3-319-93417-4_38

Shi B, Weninger T (2017) ProjE: Embedding projection for knowledge graph completion. In: Proceedings of the AAAI Conference on Artificial Intelligence, https://doi.org/10.1609/aaai.v31i1.10677

Suchanek FM, Kasneci G, Weikum G (2007) YAGO: A core of semantic knowledge. In: Proceedings of the 16th International Conference on World Wide Web, pp 697–706, https://doi.org/10.1145/1242572.1242667

Wang Q, Mao Z, Wang B, et al (2017) Knowledge graph embedding: A survey of approaches and applications. IEEE Transactions on Knowledge and Data Engineering 29(12):2724–2743. https://doi.org/10.1109/tkde.2017.2754499

Wang X, Chen L, Ban T, et al (2021) Knowledge graph quality control: A survey. Fundamental Research 1(5):607–626. https://doi.org/10.1016/j.fmre.2021.09.003

Wei J, Wang X, Schuurmans D, et al (2022) Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems, pp 24824–24837, https://doi.org/10.52202/068431-1800

White J, Fu Q, Hays S, et al (2023) A prompt pattern catalog to enhance prompt engineering with ChatGPT. In: Proceedings of the 30th Conference on Pattern Languages of Programs. ACM, https://doi.org/10.64346/PLoP2023p05

Wienand D, Paulheim H (2014) Detecting incorrect numerical data in DBpedia. In: European Semantic Web Conference, pp 504–518, https://doi.org/10.1007/978-3-319-07443-6_34

Xu L, Hu H, Zhang X, et al (2020) CLUE: A Chinese language understanding evaluation benchmark. In: Proceedings of the 28th International Conference on Computational Linguistics. International Committee on Computational Linguistics, pp 4762–4772, https://doi.org/10.18653/v1/2020.coling-main.419, URL https://aclanthology.org/2020.coling-main.419/

Yang B, Yih Wt, He X, et al (2015) Embedding entities and relations for learning and inference in knowledge bases. In: International Conference on Learning Representations, URL https://arxiv.org/abs/1412.6575

Yao S, Zhao J, Yu D, et al (2023) ReAct: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations, URL https://arxiv.org/abs/2210.03629

Zaveri A, Rula A, Maurino A, et al (2016) Quality assessment for linked data: A survey. Semantic Web 7(1):63–93. https://doi.org/10.3233/sw-150175

Zhu Z, Zhang Z, Xhonneux LP, et al (2021) Neural Bellman-Ford networks: A general graph neural network framework for link prediction. In: Advances in Neural Information Processing Systems, pp 29476–29490, URL https://papers.nips.cc/paper_files/paper/2021/hash/f6a673f09493afcd8b129a0bcf1cd5bc-Abstract.html

表 13 注入缺陷输入上的四种设置。直接调用模型使用不同提示。

变体	修复率	过度修复	F1	精确匹配
无模型补全	0.3200	0.0000	0.8472	0.0622
直接调用模型（独立提示）	0.9244	0.0758	0.9520	0.8267
无约束门控	0.9800	0.0227	0.9895	0.9333
诊断＋门控	**0.9800**	**0.0167**	**0.9917**	**0.9333**

A 补充实验

本节给出早期方法结果、过滤器和选择器的完整测试、首次收据测试、解析细节与补充检查。主要结果见第 5 节。

A.1 早期方法与过滤测试

表 13 比较四种已保存设置。不使用模型补全时返回清理后的图。直接调用模型 使用自己的提示，并跳过清理。过滤测试对同一建议在检查前后计分。第 5.3 节测试提示文本，第 A.2 节测试过滤器和选择器的各个步骤。

模型补全恢复大多数缺失字段。诊断＋门控 的 F1 为 99.17%，直接调用模型 为 95.20%。过滤保持相同修复率，并使过度修复降低 0.60 个百分点。它去掉八条未通过来源匹配的三元组，均与参考不同。

A.2 过滤器与选择器步骤测试

我们使用共享提示测试中的 360 个 Gemma 响应，以及新收据测试中的 180 个响应。每个响应经过八种过滤设置：全部检查、分别去掉五项检查中的一项、同时去掉重复与字段数量检查，以及去掉全部检查。540 个响应得到 4,320 个评分输出，候选顺序保持固定。同时去掉两项检查，用于测试两者如何相互影响。

表 14 给出完整过滤器的 F1 减去改变后过滤器的 F1。先对同一文档的三个提示取平均，再对文档取平均。完整过滤器拒绝 28 条三元组，其中 11 条文档编号错误，17 条来源匹配失败，均与参考不同。其他检查未额外去掉候选。模型构建输入中，来源匹配增加 0.505 个 F1 百分点，编号检查增加 0.085 个百分点。21 项校正后的检验 p 值均大于 0.05。日志也列出移除某项检查后丢失的正确事实。

当前编辑选择器测试。

我们在 225 个注入缺陷输入和 225 个模型构建输入的保存建议上运行 v2 选择器。13 种设置在默认绝对先验评分下产生 5,850 个结果，再用相对等权重评分重复测试，得到另外 5,850 个结果。两次均使用 `public-profile-v2` 特征和重新训练的路由器。档案保留较早的 v1 运行。

基础设置使用等权重，并让选择器始终开启。每个测试改变一个指定步骤。触发器测试从始终开启的学习权重设置开始。去掉某项分数时，将该项置零，并去掉对应边界，其他权重不变。规则检测器和违规减少奖励继续使用。去掉规则候选时，移除检测器动作和直接来源字段添加；图评估概况与保存的模型建议继续使用。

默认等权重选择器在注入缺陷输入上的 F1 为 84.72%，接受零项编辑。去掉编辑成本后，F1 提高到 89.30%，接受 68 个动作。模型构建输入上，等权重达到

表 14 各项过滤检查带来的 F1 变化，单位为百分点。每组输入有 60 篇文档和三种提示。正值表示完整过滤器更好。

移除的检查	受控输入	模型构建输入	收据
重复	+0.000	+0.000	+0.000
文档头节点	+0.037	+0.085	+0.000
关系类型	+0.000	+0.000	+0.000
来源支持	+0.037	+0.505	+0.060
字段数量	+0.000	+0.000	+0.000
重复＋字段数量	+0.000	+0.000	+0.000
全部检查	+0.073	+0.590	+0.060

表 15 默认绝对先验评分下的 v2 选择器。接受数统计动作或编辑包，按注入缺陷／模型构建输入排列。每项 F1 使用 225 篇文档。

设置	受控 F1	模型构建 F1	接受数
始终开启、均匀先验	0.8472	0.8833	0 / 10
学习先验、始终开启	0.8472	0.8799	0 / 10
学习先验与触发器	0.8472	0.8799	0 / 10
无先验惩罚	0.8472	0.8833	0 / 10
无动作成本	0.8930	0.8813	68 / 13
无密度上界	0.8472	0.8833	0 / 10
无质量下界	0.8472	0.8833	0 / 10
仅一轮	0.8472	0.8833	0 / 10
无规则候选	0.8472	0.8833	0 / 9
仅规则候选	0.8472	0.8799	0 / 10
无局部分数及边界	0.8472	0.8799	0 / 0
无图分数及边界	0.8472	0.8833	0 / 10
无来源分数及边界	0.8472	0.8799	0 / 10

88.33% F1，学习权重和学习触发器为 87.99%。两组中，各设置均保留全部原本正确的参考事实。公开文件包含文档分数、试改编辑、拒绝原因和停止原因。每种先验评分形式将 24 项组件检验作为一组进行 Holm 校正。

路由器训练与先验评分。

v2 路由器采用 $8 \to 32 \to 16$ 网络，有一个修复输出和三个编辑类型输出，共 884 个参数。特征缩放器使用 2,098 条训练记录。验证集与测试集各有 450 条记录，按文档分组。注入缺陷输入上的修复分类和编辑类型分类均达到 100%。训练标签来自已知注入缺陷。

表 16 给出较早 v2 测试中的四种权重与评分设置，上面的完整运行已包含它们。学习权重下，相对评分将注入缺陷 F1 从 84.72% 提高到 89.29%，模型构建 F1 从 87.99% 降到 87.64%。两个等权重设置返回相同的图，绝对评分继续作为默认设置。

表 16 保存建议上的 v2 选择器。相对评分以等权重为零点。接受数按注入缺陷／模型构建输入排列。

先验／偏移	受控 F1	模型构建 F1	接受数
均匀／绝对	0.8472	0.8833	0 / 10
均匀／相对	0.8472	0.8833	0 / 10
学习／绝对	0.8472	0.8799	0 / 10
学习／相对	0.8929	0.8764	156 / 51

表 17 首次 SROIE 测试，共 60 张收据。精确匹配使用有标签字段。规范化 F1 忽略取值中的大小写和空白。

方法	三元组 F1	整图匹配	规范化 F1	事实保留
抽取输入	0.7944	0.3333	0.8153	1.0000
仅规则	0.7944	0.3333	0.8153	1.0000
来源字段复制	0.0000	0.0000	0.0000	0.0000
基础，未过滤	0.7944	0.3333	0.8153	1.0000
基础，过滤后	0.7915	0.3333	0.8123	0.9944
诊断，未过滤	0.7944	0.3333	0.8153	1.0000
诊断，过滤后	0.7915	0.3333	0.8123	0.9944
SHACL 上下文，未过滤	0.7986	0.3333	0.8194	1.0000
SHACL 上下文，过滤后	0.7956	0.3333	0.8165	0.9944

A.3 收据设置与初次测试

我们使用 SROIE 收据文本和人工字段标签(Huang et al. 2019)。从修正版 trainval 镜像的 626 篇文档中随机抽取 60 篇，种子为 20260922。文本行按保存顺序连接。模型获得该文本和 company、date、address、total 四个字段名，评分器读取字段标签。一张收据没有地址标签，因此共评分 239 个字段。每种方法都在这些字段上计分；模型仍获得全部四个字段名。

Gemma 首先抽取图。三个修复提示获得同一张图，设置见第 4.4 节。样本和设置在模型调用前固定。测试包含 60 次抽取调用和 180 个修复输出，每个修复响应在过滤前后计分。我们也按原设置运行来源复制和规则修复。主分数使用精确三元组，第二种分数忽略值中的大小写和空白。

输入 F1 为 79.44%。过滤后，基础和错误报告提示均为 79.15%，SHACL 上下文为 79.56%。错误报告减基础提示的差为 0.00 个百分点（95% 置信区间：$[0.00, 0.00]$；配对随机化 $p = 1.0000$）。字段复制在收据上的 F1 为 0.00%，在明确命名字段的中文记录上为 100%。

239 个参考值中，220 个在源文本中精确出现，其余 19 个与来源字符串不同。过滤拒绝 3 个候选，全部匹配参考值。它们是在三种设置中出现的同一地址：来源有双空格，解析器却将候选中的空格压成一个。运行后，我们对来源应用相同空白规则，三种设置都保留该地址。基础与错误报告提示的 F1 变为 79.44%，SHACL 为 79.86%，整图匹配均保持 33.33%。表 17 给出原规则评分。

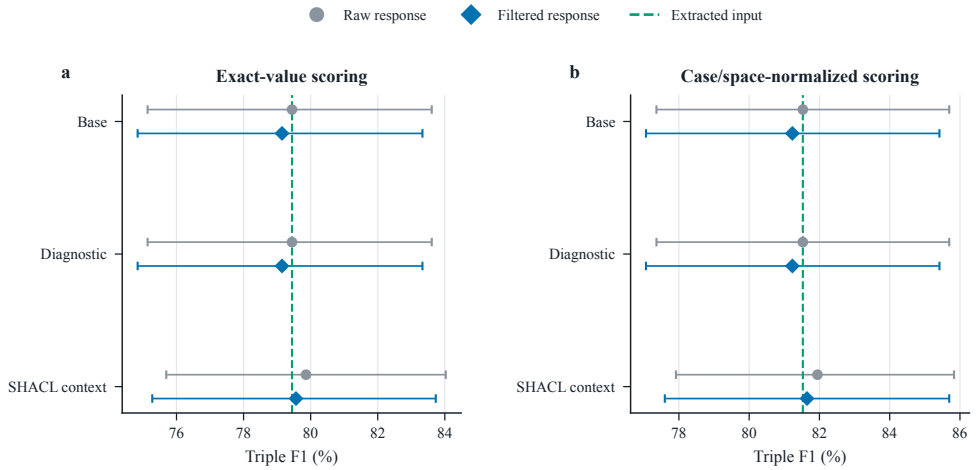


图 10 首次收据评分。点表示平均 F1，误差线为 95% 文档 bootstrap 置信区间。每个原始／过滤对照使用一个响应。虚线表示输入 F1。

在过滤前，基础提示和错误报告提示都保留全部输入图不变。49 项参考差异中，27 项涉及大小写、空格、标点或金额格式。这些发现为新收据测试中的字段含义、来源行提示和共享空白规则提供依据。

本次记录 240 次请求，包括重试，得到 240 个可解析输出。基础、错误报告和 SHACL 提示的平均修复请求时间为 7.87、7.92、8.05 秒，平均输入长度为 661、715、2276 个 token。请求时间包含间隔等待。抽取和上下文设置另计。

新收据的开发与请求。

初次 60 张收据、20 张开发收据和 60 张新测试收据的编号不同，空白清理后的源文本也不同。抽样种子为 20260923。索引设计在开发集上确定，测试标签在预测保存后下载。

开发集 F1 为：输入 80.00%，简单流程 82.50%，证据索引 85.00%，SHACL 80.00%。测试保留一个索引设计和全部三种方法。主要比较为 证据索引 减 简单流程 的文档平均 F1。代码和评分规则在测试前固定。

简单流程、证据索引、SHACL 分别修改 31、39、28 个输入图。平均修复请求时间为 7.91、7.88、8.11 秒，平均输入长度为 1784、2153、3404 个 token。240 次请求产生 60 个抽取输出和 180 个修复输出。请求时间包含间隔等待和重试。每个原始与过滤后评分对只使用一个响应。

A.4 收据修复与重新抽取细节

我们在相同的 60 张新测试收据上再收集 240 个 Gemma 响应，比较有无索引下的修复与重新抽取。各设置共享来源行、字段含义、文档编号和输出上限；修复额外获得已有图。主分数差为过滤后的索引修复减索引重新抽取。全部 240 个响应的 JSON 都包含 Markdown 代码围栏。固定的严格解析器将其记为失败，各设置 F1 均为零。

采集后，我们去掉外层围栏，并执行相同模式检查。全部 240 个响应随即可解析。表 18 给出这些补充评分。索引修复 F1 为 83.93%，索引重新抽取为 61.05%，配对差为 22.88 个百分点（95% 置信区间：14.84 至 31.35）。

表 18 60 张收据在采集后的补充评分。围栏兼容处理去掉外层 Markdown 围栏。编号规范化还在过滤前分配已知文档编号。两者均使用保存的响应。

方法	围栏兼容 F1	编号规范化 F1	错误编号图数
重新抽取	0.6333	0.7256	18/60
索引抽取	0.6105	0.7486	28/60
修复率	0.7917	0.7917	0/60
索引修复	0.8393	0.8393	0/60

索引重新抽取在 28 张图中给出错误文档编号；索引修复在全部 60 张图中使用正确编号。为四种设置统一分配已知编号后，索引重新抽取 F1 提高到 74.86%，索引修复领先 9.07 个百分点（95% 置信区间：5.22 至 13.10）。编号修正保持字段值不变。原来的 22.88 个百分点差距同时包含编号和字段值的影响。初始图构建成本与这些修复调用分别计算。

行索引覆盖的字段。
我们在来源行中定位最初后续测试的 240 个参考字段。匹配保留大小写和标点，并清理空白。一个值跨越多行时，索引必须包含某次出现的全部行。240 个值中，188 个出现在索引行或邻近行，36 个出现在范围外，16 个与来源文本不同。对于覆盖的 188 个值，简单流程 精确返回 163 个，证据索引 返回 174 个。对于范围外的 36 个值，两者都返回 26 个；与来源不同的 16 个值，两者都未返回。索引净增加的十一项精确匹配来自覆盖组。

A.5 按可见错误调用修复

我们测试基于可见编号、关系、重复、字段数量和来源匹配错误的调用规则。它为每篇文档选择返回输入，或返回保存的修复输出。在 450 项模型构建与干净图混合流上，仅对 6.22% 的输入调用修复，却漏掉 63.64% 有参考差异的图。其 F1 为 94.46%，始终调用修复为 95.82%。可见检查漏掉许多缺失或字段分配错误的值。主方法对每篇文档都调用修复。

早期选择器中的有用编辑。
我们使用早期选择器保存的建议与试改图。这些运行早于表 16 中的共享特征和候选启动修改。对每篇文档，尝试模型编辑包的所有子集，并根据参考图为结果计分。执行保存编辑时关闭停止、成本和图边界检查。参考得分最高的子集称为 oracle。这项分析使用模型编辑包和参考值。

我们在 450 个输入上尝试 2,144 个子集，结果见表 19。注入缺陷输入中，86 张图未检测到违规，其中 72 张缺少参考字段，70 张存在能提高 F1 的建议子集。检查的试改中，212 项提高参考 F1，但效用分数不大于零。模型构建输入的对应数量为 8 张图和 1 项试改。

去掉编辑成本使 67 篇注入缺陷文档的 F1 提高，158 篇不变，0 篇降低。模型构建输入中，0 篇提高，222 篇不变，3 篇降低。这三张图在清理后已经与参考不同。全部 148 个原本精确匹配的模型构建输入保留其 F1。注入缺陷输入的配对增益为

表 19 早期选择器结果，每组 225 个输入。oracle 利用参考答案寻找最佳保存子集。一篇文档可以有多次试改。

诊断项	受控输入	模型构建输入
预处理图 F1	0.8472	0.8799
均匀选择器 F1	0.8472	0.8833
固定建议 oracle F1	0.9964	0.9173
初始未检出违规	86	198
其中与参考不同的图	72	50
其中存在可改善结果的图	70	8
提高参考分数但效用非正的试改	212	1

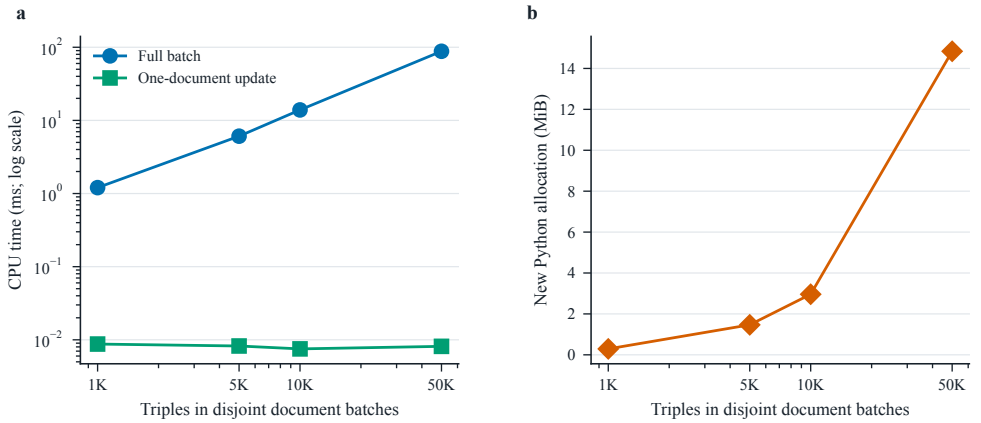


图 11 独立文档批次的报告时间和内存。局部更新只检查一篇修改后的文档。内存统计共享源文本时新增的批次存储。

4.58 个百分点（95% 文档 bootstrap 置信区间：3.69–5.54）。同一成本修改在两组输入上作用不同。

A.6 图检查的时间与内存

我们在包含 1K–50K 条三元组的独立文档批次上，测量式 (2) 中报告的耗时。完整扫描检查所有文档，局部更新只检查一篇已修改文档。完整扫描时间取七次运行的中位数，局部更新时间取 200 次运行的中位数，数值见表 21。内存统计构建批次时新增 Python 内存的峰值。源文本和模式对象共享，模型调用时间另计。

A.7 补充质量与抽取检查

Gemma 评判器在方法名称隐藏的条件下，为 180 个来源与三元组对打分。来源分数与精确参考有效性的 Pearson 相关系数为 $r = 0.794$，Spearman 相关系数为 $\rho = 0.807$（两者均有 $p < 2.4 \times 10^{-40}$）。另外五次运行中，固定 50 项样本的均值都为 0.96。

表 20 请求成本。平均时间包含重试和请求间隔等待。来源设置与排队时间另计。

输入	上下文	调用数	输入 token	输出 token	总耗时（秒）
受控输入	基础提示	1.00	947	517	8.58
受控输入	诊断提示	1.00	1017	512	8.61
受控输入	SHACL	1.00	3080	513	8.45
模型构建输入	基础提示	1.00	832	458	7.51
模型构建输入	诊断提示	1.00	901	462	7.90
模型构建输入	SHACL	1.00	2822	455	8.16

表 21 图报告的时间和内存。内存统计新增批次存储峰值，单位为 MiB，源文本共享。

三元组	文档数	完整（毫秒）	局部（毫秒）	内存（MiB）
1,000	143	1.20	0.009	0.29
5,000	716	6.08	0.008	1.47
10,000	1,434	13.90	0.008	2.96
50,000	7,173	88.10	0.008	14.84

我们将本文抽取器与 Neo4j Labs 的 Graph Builder 使用的 `LLMGraphTransformer` 核心(Neo4j Labs 2024)比较，测试使用 CLUE 中的 45 篇均衡 TNEWS 文档(Xu et al. 2020)。字段抽取器的类别准确率为 0.556，Graph Builder 为 0.533，配对检验为 $p = 1.00$。

A.8 请求成本与检查时间

译文附页：图中文字对照

本页对照原图中的英文标注。图形、数据点、颜色和数值沿用英文论文。方法名称在正文和表格中采用以下中文写法。

原图方法名	中文名称
Source-field Copy	来源字段复制
No Repair	不修复
Rule Only	仅规则
SHACL-style	SHACL 风格修复
Direct LLM	直接调用模型
ReAct-style	ReAct 风格智能体
Simple Pipeline / Simple	简单流程
Diagnosis + Gate	诊断＋门控
Evidence Index / Index	证据索引
Base	基础提示
Diagnostic	诊断提示
SHACL context	SHACL 上下文

图 1：三个检查范围。

- 输入：One document · three views 为“一篇文档、三个视角”；Source X 为“原文 X”；Task schema 为“任务模式”；Head 为“头节点”；Relations 为“关系”。示例 Merchant: ALPHA、Total: 42.00 表示“商户：ALPHA、总额：42.00”。
- Local scope 为“局部范围”；Incidence + duplicates 为“节点连接与重复”；merchant 为“商户”；isolated node 为“孤立节点”；Repeated fact → duplicate flag 为“事实重复，标记为重复”；No incident edge → local flag 为“没有关联边，标记为局部问题”；Scores: incidence, uniqueness 为“分数：节点连接、唯一性”。
- Graph scope 为“图范围”；Head + relation schema 为“头节点与关系模式”；seller 为“卖方”；seller is not permitted 为“seller 关系不在允许名单内”；$H(t)$ 的说明为“头节点匹配 d1”；$R(t)$ 的说明为“关系被允许”；Score: rule consistency 为“分数：规则一致性”。
- Source scope 为“来源范围”；Value ↔ source text 为“值与源文本对应”；Source excerpt 为“来源片段”；Graph value: 49.00 → absent 为“图中取值 49.00 在原文中不存在”；$S(t; X)$ 的说明为“非空值出现在原文中”；Score: source support 为“分数：来源支持”。底部 Shared report 表示“共享报告：重复计数、失败检查、已出现的关系”。

图 2：路由与编辑选择。

- Input graph 为“输入图”；Document graph G_t 为“文档图 G_t”；Source X 为“原文 X”。示例 merchant、total 分别为“商户、总额”，图中的 49.00 对应来源中的 42.00。
- Quality profile 为“质量评估概况”；Four quality scores 为“四项质量分数”：Incidence、Uniqueness、Rule consistency、Source support，分别为

“节点连接、唯一性、规则一致性、来源支持”。Four graph statistics 为“四项图统计量”：Nodes、triples、Density、violations，即“节点数、三元组数、密度、违规数”。

- Neural routing 为“神经路由”；8 inputs → router 为“八个输入进入路由器”；Repair trigger 为“修复触发分数”；Scope prior π 为“范围先验 π”；Local、Graph、Source 为“局部、图、来源”。Hard flags or candidates override a low trigger 表示“存在硬规则标记或候选时，较低触发分数仍会启动试改”。
- Edit selection 为“编辑选择”；Candidate edits 为“候选编辑”；Fixed model bundles + rule-derived proposals 为“固定模型编辑包加规则建议”；Delete、Retype、Add 分别为“删除、改关系、添加”。Trial graphs 为“试改图”；Bounds + utility $U(a)$ 为“边界与效用 $U(a)$”。选择步骤为“选择可执行且 $U(a) > 0$ 最高的一项编辑”。
- Commit one edit → G_{t+1} 为“执行一项编辑，得到 G_{t+1}”；随后“保留决策日志，重新评估接受后的图”。底部说明：顺序变体在每次接受编辑后重新评估；没有可执行的正效用编辑时停止。

图 3：一次试改循环。

- Route 为“路由”：当前图 G_t 输入四项评估分数和四项图统计量。Learned router 为“学习得到的路由器”；输出修复分数 p(repair) 和范围先验 π。存在硬规则违规或已提供候选时，低触发分数仍继续试改。
- Propose 为“提出候选”：Model proposals 为“模型建议”；Fixed edit bundles grouped by relation 为“按关系分组的固定编辑包”；Rule proposals 为“规则建议”；Detected violations + explicit source fields 为“检测到的违规与明确来源字段”。动作是删除、改关系和添加；Deduplicate candidates 为“对候选去重”。
- Evaluate trials 为“评估试改”：在 G_t 的独立副本上分别尝试 a_1, a_2, a_3，得到试改图。Recompute profile and flags 为“重新计算评估概况与标记”；Check bounds; calculate $U(a)$ 为“检查边界，计算 $U(a)$”。
- Select 为“选择”：先判断是否允许执行、是否满足 $U(a) > 0$，再在合格编辑中选择最高效用，执行一项编辑，使 G_t 变为 G_{t+1}。
- Utility $U(a)$ 的文字说明为“质量增益＋硬规则违规减少－编辑成本＋对数范围先验＋建议置信度”。Trial-state constraints 为“试改状态约束”，包括“恢复边界、单项分数有限回退、密度上界、非空图保护”。
- 底部说明：重新评估执行后的图，并在 12 步范围内重复；没有合格编辑或达到 12 步时停止。提前退出条件为“没有已提供候选，且低触发分数下没有硬规则标记，或没有检测到违规”。

图 4：受控修复。

a 中 Quality with 95% bootstrap intervals 为“质量分数及 95% bootstrap 区间”；Defect repair、Triple F1、Exact graph 为“缺陷修复、三元组 F1、整图匹配”；Held-out score 为“预留测试得分”。b 中 Quality–latency trade-off 为“质量与耗时的权衡”；纵轴为“缺陷修复率”，横轴为“每篇文档耗时（秒）”。圆形和方形表示一次调用，菱形表示两次调用。方法名见本附页开头。

图 5：按输入类型分组。

Extracted input 为“抽取输入”；Simple pipeline 为“简单流程”；All inputs 为“全部输入”；Valid JSON 为“有效 JSON”；Imperfect valid JSON 为“有效但与参考不同的 JSON”；Malformed JSON 为“无效 JSON”。n 表示该组样本数。a 的横轴为“三元组 F1（%）”，b 为“整图精确匹配（%）”。其他方法名见本附页开头。

图 6：机制对照。

Raw candidate 为“原始候选”；Gated pipeline 为“门控过滤流程”；Sequential optimizer 为“顺序选择器”。a 为“受控输入”，b 为“模型构建输入”，c 为“保存的候选”；Archived candidates 表示“历史保存候选”。三幅图横轴均为“三元组 F1（%）”。Base、Diagnostic、SHACL context 的对应中文见开头。

图 7：新收据。

Extracted input 为“抽取输入”，a 的横轴为“三元组 F1（%）”。b 的三项为“简单流程减输入”“索引减简单流程”“索引减 SHACL”，横轴 Paired F1 difference (points) 为“配对 F1 差（百分点）”。方法名见开头。

图 8：析因与来源行。

a 的 Factorial main effects 为“析因主效应”；Preprocessing、Diagnosis、Gate 分别为“预处理、诊断、门控”；Controlled、Natural 分别为“受控输入、模型构建输入”。b 的 Receipt context contrasts 为“收据上下文对照”；Full、Random、Anchors 分别为“完整索引、随机索引、仅锚点”；No-def. full 为“无字段定义的完整索引”，No-def. simple 为“无字段定义的简单流程”。各项依次比较“完整减简单”“完整减随机”“完整减锚点”“完整减无定义完整”“简单减无定义简单”。横轴均为“配对 F1 变化（百分点）”。

图 9：合同。

Initial extraction、Simple repair、Indexed repair、Indexed re-extraction 分别为“初始抽取、简单流程修复、索引修复、索引重新抽取”。横轴 Document mean F1 (%) with 95% bootstrap interval 为“文档平均 F1（%）及 95% bootstrap 区间”。

图 10：首次收据。

Raw response、Filtered response、Extracted input 分别为“原始响应、过滤后响应、抽取输入”。a 的 Exact-value scoring 为“精确值评分”，b 的 Case/space-normalized scoring 为“大小写与空白规范化评分”。横轴为“三元组 F1（%）”。提示名称见本附页开头。

图 11：检查规模。

Full batch 为“完整批次”；One-document update 为“单文档更新”。a 的纵轴 CPU time (ms; log scale) 为“CPU 时间（毫秒，对数坐标）”。b 的纵轴 New Python allocation (MiB) 为“新增 Python 内存分配（MiB）”。两幅图横轴 Triples in disjoint document batches 为“互相独立的文档批次中的三元组数”，1K、5K、10K、50K 分别表示一千、五千、一万和五万条。